Lecture 5.1

Image Segmentation

Global Thresholding, Adaptive / Local Thresholding

Instructor Name:

Prof. Dr. Jahangir Alam

Dept. of CSE, BAUST

Image Segmentation using Thresholding

Image Segmentation is a fundamental task in **image processing and computer vision** where an image is divided into meaningful regions (segments) to simplify analysis.

It is the process of partitioning an image into multiple regions such that:

- Pixels in the same region share similar properties (color, intensity, texture).

- Different regions represent different objects or parts of objects.

Goal: Convert an image into something easier to analyze.

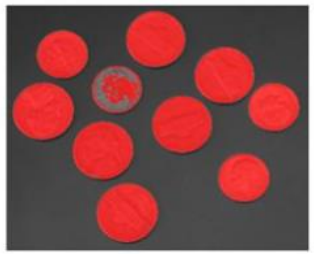
Image Segmentation using Thresholding

Goals of Segmentations:

- Identify objects or Isolate regions of interest (ROI).
- Simplify the image representation for further analysis.
- Enable feature extraction, classification, or diagnosis.







Applications

- Medical imaging (tumor detection)
- Autonomous driving (road, pedestrians)
- Satellite imaging (land cover classification)
- Face recognition
- Crop disease detection

Fundamentals

Let R represent the entire spatial region occupied by an image. We may view image segmentation as a process that partitions R into n subregions, $R_1, R_2, \ldots, R_n$, such that

(a) $\bigcup_{i=1}^{n} R_i = R.$

(b) R_i is a connected set, $i = 1, 2, \ldots, n.$

(c) $R_i \cap R_j = \varnothing$ for all i and j, $i \neq j.$

(d) $Q(R_i) = \text{TRUE}$ for $i = 1, 2, \ldots, n.$

(e) $Q(R_i \cup R_j) = \text{FALSE}$ for any adjacent regions R_i and R_j.

Here, $Q(R_k)$ is a logical predicate defined over the points in set R_k, and $\varnothing$ is the null set. The symbols $\cup$ and $\cap$ represent set union and intersection, respectively, as defined in Section 2.6.4. Two regions R_i and R_j are said to be *adjacent* if their union forms a connected set, as discussed in Section 2.5.2.

Conceptual Segmentation Workflow

Input Image

↓

Preprocessing

(Noise Reduction, Contrast Enhancement)

↓

Segmentation

- Thresholding
- Edge / Region-based/Clustering/CNN
- ML / DL

↓

Postprocessing

(Morphological cleaning, smoothing)

↓

Feature Extraction

(Area, Shape, Texture, Intensity)

↓

Classification / Diagnosis

Performance Metrics

Metric	Meaning	
Dice Coefficient (DSC)	$DSC = \frac{2\lvert A \cap B \rvert}{\lvert A \rvert + \lvert B \rvert} = \frac{2IoU}{1 + IoU}$ $= 2\frac{2TP}{TP + FP + FN}$	$A \cap B$
Jaccard Index (IoU)	$IoU = \frac{\lvert A \cap B \rvert}{\lvert A \rvert + \lvert B \rvert - \lvert A \cap B \rvert} = \frac{DSC}{2 - DSC}$ $= \frac{TP}{TP + FP + FN}$	$A \cap B$
Accuracy	Correct pixels / Total pixels $Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$	Overall correctness
Precision & Recall	For object vs background pixels $Precision = \frac{TP}{TP + FP}, Recall = \frac{TP}{TP + FN}$	Quality of segmentation

Types of Segmentation Methods

- **Threshold-based Segmentation** (Global / Otsu / Adaptive)
- **Edge-based Segmentation** (Canny, Sobel, etc.)
- **Region-based Segmentation** (Region growing, merging, split-merge)
- **Clustering-based Segmentation** (K-Means, Fuzzy C-Means)
- **Neural Network / Deep Learning Segmentation** (U-Net, CNN, etc.)
- **Watershed Segmentation**

Image Segmentation using Thresholding

It helps computers understand "what is where" in an image — for example:

- In medical imaging (like CT, MRI, X-ray, USG), segmentation helps isolate **organs, tumors, bones** for analysis and diagnosis.

- In satellite images → separating **land, water, vegetation**.

- In general vision → separating **object from background**.

Threshold-Based Segmentation

Simplest and fastest. Separate objects based on pixel intensity.

a) Global Thresholding:
Global Thresholding is a segmentation technique that converts a grayscale image into a binary image using a single threshold value (T) for the entire image.

Goal: Separate foreground (object) from background in an image.

Steps in Global Thresholding

Every input pixel intensity $f(x, y)$ is compared with a global threshold T:

1. **Input:** Grayscale image $f(x, y)$
2. **Compute histogram:** Get pixel intensity distribution.
3. **Select threshold T:**

- Manually, or
- Automatically using a method like **Automatic Threshold (Iterative/ Isodata Method)/ Otsu's algorithm**.

1. **Apply threshold rule:** Convert pixel values using one threshold T divides pixels into two groups:

$$g(x, y) = \begin{cases} 1 & for\ f(x, y) > T \\ 0 & for\ f(x, y) \leq T \end{cases}$$

1. **Output:** Binary image with object and background separated.

Global Thresholding

Automatic Threshold (Iterative/ Isodata Method)

1. Start with an initial intensity (0 to $L-1$) threshold $T_0 = \frac{I_{min}+I_{max}}{2}$ (e.g., mean intensity).
2. Separate pixels into:

- G_1: pixels > T_0
- G_2: pixels ≤ T_0

1. Compute the **mean intensity** of each group:

$$\mu_1 = mean(G_1),\ \mu_2 = mean(G_2)$$

1. Update threshold:

$$T_{k+1} = \frac{\mu_1+\mu_2}{2}$$

1. Repeat steps 2–4 until the threshold value converges:

$$| T_{k+1} - T_k | < \epsilon\ (small \text{ value}).$$

Global Thresholding

Example/Applications:

- ✓ Document binarization (text extraction)
- ✓ Bone (bright) vs soft tissue (dark) in X-ray.
- ✓ separate bone (bright) from tissue (dark) in an X-ray.
- ✓ Segmenting bones from X-ray (bones are brighter → higher intensity).
- ✓ Object detection in industrial inspection

Threshold-Based Segmentation (Binary)

$$Original\ Image = \begin{bmatrix} 120 & 130 & 140 \\ 100 & 150 & 160 \\ 90 & 110 & 170 \end{bmatrix}$$

Choose a Threshold Value

Let's use a **threshold = 130**

- If a $pixel\ value \geq 130 \rightarrow$ assign it **1** (white, foreground)
- If a $pixel\ value < 130 \rightarrow$ assign it **0** (black, background)

Apply Thresholding

We create a **binary segmented image**:

$$Segmented\ Image = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

Original Value	Thresholded (Segmented)
120	0
130	1
140	1
100	0
150	1
160	1
90	0
110	0
170	1

Threshold-Based Segmentation (Binary)

$$Image = \begin{bmatrix} 40 & 45 & 50 & 200 \\ 42 & 46 & 55 & 210 \\ 44 & 48 & 60 & 220 \\ 43 & 47 & 52 & 230 \end{bmatrix}$$

Let's choose a threshold T=100

- If pixel $\geq$ T $\rightarrow$ Foreground (1)
- If pixel $<$ T $\rightarrow$ Background (0)

Threshold-Based Segmentation (Binary)

Consider a small **4×4 grayscale image** with pixel intensities from 0 to 255:

$$I = \begin{bmatrix} 10 & 50 & 200 & 210 \\ 20 & 60 & 190 & 220 \\ 15 & 55 & 180 & 230 \\ 25 & 65 & 170 & 240 \end{bmatrix}$$

Objective:

Segment the image into **two regions**:

- **Background:** pixels with intensity **less than or equal to 100**
- **Foreground (object):** pixels with intensity **greater than 100**

Choose Threshold T=100

Apply Thresholding Rule

$$S(x,y) = \begin{cases} 0 & if\ I(x,y) \leq 100 (Background) \\ 1 & if\ I(x,y) > 100\ (Foreground) \end{cases}$$

Threshold-Based Segmentation (Binary)

Consider a small **4×4 grayscale image** with pixel intensities from 0 to 255:

$$I = \begin{bmatrix} 10 & 50 & 200 & 210 \\ 20 & 60 & 190 & 220 \\ 15 & 55 & 180 & 230 \\ 25 & 65 & 170 & 240 \end{bmatrix} \Rightarrow S = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

Create Segmented Image *S*

Apply threshold to each pixel:

Pixel	(0,0)	(0,1)	(0,2)	(0,3)	(1,0)	(1,1)	(1,2)	(1,3)	(2,0)	(2,1)	(2,2)	(2,3)	(3,0)	(3,1)	(3,2)	(3,3)
Intensity	10	50	200	210	20	60	190	220	15	5	180	230	25	65	170	240
$S(x, y)$	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1

Interpretation

- Pixels labeled **1** correspond to the **foreground object**.
- Pixels labeled **0** correspond to the **background**.

Global Thresholding (Finger Print Processing)

Target:

Segment a fingerprint image into:

•Ridges (dark = background)

•Valleys (light = foreground)

Example:

Let's assume a 5×5 grayscale block extracted from a fingerprint image:

35	**45**	**48**	**44**	**30**
38	50	52	46	32
40	55	70	58	36
39	51	60	49	35
33	47	53	42	28

Step 1: Choose Threshold T

Let's start with initial threshold $T_0 = 45$ (can be mean or estimated).

Global Thresholding (Finger Print Processing)

35	45	48	44	30
38	50	52	46	32
40	55	70	58	36
39	51	60	49	35
33	47	53	42	28

Step 2: Apply Thresholding

Compare each value to 45:

$$g(x,y) = \begin{cases} 1 & if\ f(x,y) \geq 45 \\ 0 & if\ f(x,y) < 45 \end{cases}$$

This binary image highlights ridges/valleys for further processing.

0	1	1	0	0
0	1	1	1	0
0	1	1	1	0
0	1	1	1	0
0	1	1	0	0

Global Thresholding (Finger Print Processing)

35	45	48	44	30
38	50	52	46	32
40	55	70	58	36
39	51	60	49	35
33	47	53	42	28

Step 3: (Optional) Update Threshold Iteratively

- Separate into two groups:
  - ✓ G1: pixels ≥ T: {45, 48, ..., 70, ...}
  - ✓ G2: pixels < T: {35, 30, 38, ...}
- Compute new means:
  - ✓ $\mu_1 \approx 52.5$
  - ✓ $\mu_2 \approx 35.2$
- New threshold:

$$T1 = \frac{\mu_1 + \mu_2}{2} = \frac{52.5 + 35.2}{2} \approx 43.9$$

Repeat until threshold stabilizes (used in iterative global thresholding).

Global Thresholding (Finger Print Processing)

35	45	48	44	30
38	50	52	46	32
40	55	70	58	36
39	51	60	49	35
33	47	53	42	28

Applications in Fingerprint:

- Segments ridges (dark) vs background
- Prepares image for thinning, edge detection, or minutiae extraction

Summary:

Step	Description
1	Estimate threshold (e.g., average)
2	Apply threshold to binarize image
3	(Optional) Recalculate threshold
4	Output: binary image for fingerprint

Adaptive / Local Thresholding

Threshold $T(x, y)$ changes over local neighborhoods (good for uneven illumination). Threshold varies by region to handle non-uniform lighting.

Example:

Input: [40, 45, 150, 160, 170]

T = 100

Result: [0, 0, 1, 1, 1]

→ 0 = background, 1 = object.

Threshold $T(x, y)$ changes locally for uneven lighting.

In image segmentation, adaptive thresholding determines the threshold locally using statistics like mean or Gaussian-weighted mean in a window (e.g., 3×3 or 5×5), instead of one global value.

This helps segment objects when lighting varies across the image — common in document images, fingerprints, or X-rays.

Adaptive / Local Thresholding

Instead of using a single global threshold T, adaptive thresholding computes a local threshold for each pixel based on its neighborhood (local window).

Formula (Mean Method):

$$g(x,y) = \begin{cases} 1 & if\ f(x,y) \geq \mu(x,y) - C \\ 0 & Otherwise \end{cases}$$

- $f(x,y)$: input grayscale pixel
- $\mu(x,y)$: mean of neighborhood around (x,y)
- C: constant to fine-tune threshold (e.g., 5)

Example1: Adaptive / Local Thresholding

Example (3×3 Block Neighborhood)
Let's assume this 5×5 grayscale image:
Let's threshold the center pixel (2,2) = 55
using a 3×3 window and C=5

45	50	55	60	65
40	50	60	70	80
35	45	55	65	75
30	40	50	60	70
25	35	45	55	65

Step 1: Get 3×3 Neighborhood of (2,2)

45	50	55
40	50	60
35	45	55

Step 2: Compute Local Mean

$$\mu(2,2) = \frac{45 + 50 + 55 + 40 + 50 + 60 + 35 + 45 + 55}{9} = 48.3$$

Example1: Adaptive / Local Thresholding

Example (3×3 Block Neighborhood)
Let's assume this 5×5 grayscale image:
Let's threshold the center pixel (2,2) = 55
using a 3×3 window and C=5

45	50	55	60	65
40	50	60	70	80
35	45	55	65	75
30	40	50	60	70
25	35	45	55	65

Step 3: Apply Threshold Rule

$$T = \mu - C = 48.3 - 5 = 43.3$$

Now compare center pixel (55):

$$55 > 43.3 \Rightarrow Output = 1$$

Example1: Adaptive / Local Thresholding

45	50	55	60	65
40	50	60	70	80
35	45	55	65	75
30	40	50	60	70
25	35	45	55	65

Result:

•The center pixel becomes white (1) in the binary image.

Repeat for Other Pixels:

You can slide the 3×3 window and apply the same steps to each pixel.

Summary:

Step	Operation
1	Choose window size (e.g. 3×3)
2	Compute mean of local pixels
3	Subtract constant C
4	Threshold current pixel

Why It's Better?

Global Thresholding	Adaptive Thresholding
One threshold for all	Different threshold per pixel
Fails in uneven lighting	Robust to shadows/illumination
Fast but less accurate	Slightly slower but better

Example2: Adaptive / Local Thresholding

Let's work with a 5×5 grayscale image:
We will apply adaptive mean thresholding using:

- Window size = 3×3
- C = 5 (a constant subtracted from local mean)

40	42	44	46	48
38	40	42	44	46
36	38	40	42	44
34	36	38	40	42
32	34	36	38	40

Step 1: for Pixel at (2,2) = 40
Neighborhood (3×3 around (2,2)):

40	**42**	**44**	46	48
38	**40**	**42**	44	46
36	**38**	**40**	42	44
34	36	38	40	42
32	34	36	38	40

Example2: Adaptive / Local Thresholding

Let's work with a 5×5 grayscale image:
We will apply adaptive mean thresholding using:
•Window size = 3×3
•C = 5 (a constant subtracted from local mean)

40	**42**	**44**	46	48
38	**40**	**42**	44	46
36	**38**	**40**	42	44
34	36	38	40	42
32	34	36	38	40

Step 3: Apply Threshold Rule
Threshold, $T = \mu - C = 40 - 5 = 35$

$$40 > 35 \Rightarrow Output = 1(white)$$

- The center pixel becomes white (1) in the binary image.

Step 2: Compute Local Mean: Mean of 3×3 neighborhood,

$$\mu = \frac{40 + 42 + 44 + 38 + 40 + 42 + 36 + 38 + 40}{9} = 40$$

Example2: Adaptive / Local Thresholding

40	**42**	**44**	46	48
38	**40**	**42**	44	46
36	**38**	**40**	42	44
34	36	38	40	42
32	34	36	38	40

Repeat for Other Pixels:
We apply this to the inner 3×3 region of the image (excluding border pixels). Here's the result for each:

$40 > 35$

Pixel (i,j)	Value	Local Mean	T=Mean−C	Output
(1,1)	40	40	35	1
(1,2)	42	41.11	36.11	1
(1,3)	44	42.22	37.22	1
(2,1)	38	39.11	34.11	1
(2,2)	40	40	35	1
(2,3)	42	41.11	36.11	1
(3,1)	36	37.11	32.11	1
(3,2)	38	38	33	1
(3,3)	40	39	34	1

Final Binary Output (Center 3×3 Region):

1	**1**	**1**
1	**1**	**1**
1	**1**	**1**

Example2: Adaptive / Local Thresholding

Conclusion:

- ✓ Adaptive thresholding helps segment images where global thresholding would fail due to uneven lighting.

- ✓ Each pixel is binarized using its local statistics.

Autonomous Vehicles: Lane Detection

##Autonomous vehicles use image processing to perform a key task — lane detection — one of the foundational components of self-driving systems.

Target: Detect the lane lines from a road image to keep the vehicle centered.

Step 1: Input Image (Example: 5×5 grayscale road image)

We'll use a simplified image where:

- 0 = dark (black/road)
- 255 = bright (white/lane line)

Example 5×5 grayscale image matrix:

	0	0	255	0	0
	0	0	255	0	0
I=	0	0	255	0	0
	0	255	0	255	0
	255	0	0	0	255

Here, the bright pixels (255) roughly form lane-like structures.

Autonomous Vehicles: Lane Detection

Step 2: Grayscale to Binary Conversion (Thresholding)

Use a threshold (e.g., 100) to convert the image to binary:

- If pixel > 100 → 1 (white/lane)
- Else → 0 (black/background)

	0	0	1	0	0
	0	0	1	0	0
B=	0	0	1	0	0
	0	1	0	1	0
	1	0	0	0	1

Step 3: Edge Detection (Optional but common)

Use a Sobel filter to detect vertical or diagonal lane edges. For simplicity, we skip numerical convolution here, but edge detection would highlight boundaries of lane lines.

Autonomous Vehicles: Lane Detection

Step 4: Hough Transform (Conceptual)

Use the Hough Line Transform to detect lines in the binary image.

Example:

From binary image *B*, the algorithm identifies vertical lines through columns:

- Column 2 and 4 likely represent lane boundaries.

Step 5: Lane Position Estimation

Assume image width = 5 pixels.

- Detected lane lines: column 2 and 4
- Lane center = average = (2 + 4)/2 = column 3

Vehicle is already centered in this simple example

Autonomous Vehicles: Lane Detection

Step 6: Decision Making

Based on lane positions:

- If the car is left of center → steer right.
- If car is right of center → steer left.

Summary:

Step	Operation
1. Image Capture	Get road image via camera
2. Thresholding	Convert grayscale to binary
3. Edge Detection	Find lane line edges (Sobel/Canny)
4. Hough Transform	Detect straight lines
5. Position Estimation	Locate lane lines
6. Control Decision	Adjust steering accordingly

Thank You

Lecture 5.2

Image Segmentation

Gaussian Adaptive Thresholding, Local Binary Pattern (LBP)

Instructor Name:

Prof. Dr. Jahangir Alam

Dept. of CSE, BAUST

Gaussian Adaptive Thresholding

Instead of computing the mean of a local window, we compute a Gaussian-weighted sum — giving more weight to pixels closer to the center of the window.

Formula:

$$T(x,y) = \mu_{Gaussian}(x,y) - C$$

$$\Rightarrow g(x,y) = \begin{cases} 1 & if\ f(x,y) > T(x,y) \\ 0 & Otherwise \end{cases}$$

Goal: Binarize pixel values based on a local Gaussian-weighted threshold.

Gaussian Adaptive Thresholding

Let's use a 3×3 grayscale image patch centered on pixel *f(2,2)=42*:

Neighborhood:

40	41	43
39	42	44
38	40	45

Use a 3×3 Gaussian weight kernel (simplified integer approximation):

1	2	1
2	4	2
1	2	1

Total weight = 16 (sum of all weights)

Let, C=3

Step 1: Weighted Sum

$$\mu_{Gaussian}(x, y) = \frac{(1 \times 40 + 2 \times 41 + 1 \times 43) + (2 \times 39 + 4 \times 42 + 2 \times 44) + (1 \times 38 + 2 \times 40 + 1 \times 45)}{16}$$

$$\approx 41.375$$

Gaussian Adaptive Thresholding

Step 2: Compute Threshold

$$T = \mu - C = 41.375 - 3 = 38.375$$

Step 3: Compare Center Pixel to Threshold

Pixel at center $f(1,1) = 42$

$$42 > 38.375 \Rightarrow 1(white)$$

1	1	1
1	1	1
1	1	1

Final Output: Center pixel becomes white (1) in binary image.

Summary Table:

Step	Result
Neighborhood	3×3 gray values
Gaussian Weights	[1 2 1; 2 4 2; 1 2 1]
Weighted Mean	≈ 41.38
Threshold (T)	38.38
Center Pixel Value	42
Binary Output	1 (since 42 > 38.38)

Why Gaussian Is Better than Mean?

- Reduces noise by weighting closer pixels more heavily
- Preserves fine details like edges in adaptive segmentation

Gaussian Adaptive Thresholding

45	48	50	50	52
44	46	49	53	55
43	45	50	54	56
42	44	48	51	53
40	42	44	47	50

2. Gaussian Kernel (3×3) (simplified integer weights)

1	2	1
2	4	2
1	2	1

Total Weight = 16

We'll apply this kernel centered on each non-border pixel (i.e., positions (1,1) to (3,3)).

Let's set C = 3 (a common value in adaptive thresholding)

3. Thresholding Center Pixel at (2,2) = 50

Take 3×3 patch around (2,2):

46	49	53
45	50	54
44	48	51

Step 4: Weighted Gaussian kernel **Sum**

$$\mu = \frac{(1 \times 46 + 2 \times 49 + 1 \times 53) + (2 \times 45 + 4 \times 50 + 2 \times 54) + (1 \times 44 + 2 \times 48 + 1 \times 51)}{16}$$

$$= 49.125$$

Gaussian Adaptive Thresholding

Step 5: Compute and compare Threshold

$$T = 49.125 - 3 = 46.125$$

Compare center pixel (50):

$$50 > 46.125 \Rightarrow 1$$

Thus, pixel (2,2) → white (1) in the binary image.

45	48	50	50	52
44	46	49	53	55
43	45	50	54	56
42	44	48	51	53
40	42	44	47	50

	1	2	1
1/16	2	4	2
	1	2	1

Step 6: Repeat for Other Center Pixels:

46	49	53
45	50	54
44	48	51

Position	Pixel	Local Mean	T (mean - C)	Output
(1,1)	46	46.8	43.875	1
(1,2)	49	48.00	45.00	1
(1,3)	53	49.625	46.625	1
(2,1)	45	45.875	42.875	1
(2,2)	50	49.125	46.125	1
(2,3)	54	50.375	47.375	1
(3,1)	44	44.75	41.75	1
(3,2)	48	47.00	44.00	1
(3,3)	51	48.75	45.75	1

Gaussian Adaptive Thresholding

7. Final Binary Image (3×3 center part):
All center pixels remain white (1) after thresholding.

1	**1**	**1**
1	**1**	**1**
1	**1**	**1**

Result: Image Segmentation Using Gaussian Adaptive Thresholding

- Dark regions stay dark (0), light regions go white (1)
- Locally adjusts for brightness differences
- Retains features in real-world scenarios like unevenly lit fingerprints or documents.

Iris Recognition Variation of Local Binary Pattern (LBP)

Local Binary Pattern (LBP), one of the most widely used texture feature extraction techniques in image processing and computer vision.

It is a texture descriptor that encodes the local structure (edges, corners, flat areas) of an image.

It works by comparing each pixel with its neighborhood and forming a binary code.

LBP Feature Extraction Diagram

Input Image → Gray Conversion → LBP Operation → LBP Image → Histogram (Feature Vector)

Iris Recognition Variation of Local Binary Pattern (LBP)

Applications

Field	Example
Face Recognition	Detect facial textures (Fisherfaces + LBP)
Medical Imaging	Texture classification in MRI, CT, USG
Surface Inspection	Detect scratches, defects on textures
Biometrics	Palmprint, iris, fingerprint analysis
Remote Sensing	Land cover classification

Iris Recognition Variation of Local Binary Pattern (LBP)

Concept

For a pixel I_c (center pixel), compare it with its surrounding pixels I_p (neighbors).

Rule:

$$LBP_P, R = \sum_{p=0}^{P-1} s(I_p - I_c) \times 2^p$$

where

$$s(x) = \begin{cases} 1 & if\ x \geq 0 \\ 0 & if\ x < 0 \end{cases}$$

P = number of neighbors (typically 8)

R = radius of the circular neighborhood (typically 1)

Iris Recognition Variation of Local Binary Pattern (LBP)

Algorithm Steps

Step 1: Select a pixel (center)
Take a 3×3 window around it.

Step 2: Compare neighbors
Compare each of the 8 surrounding pixels with the center pixel.

Step 3: Assign binary code
If neighbor ≥ center → write 1, else 0.

Step 4: Convert binary → decimal
Form a binary number clockwise, then convert it to a decimal (0–255).

Step 5: Create LBP image
Replace each center pixel with its LBP code → texture map.

Iris Recognition Variation of Local Binary Pattern (LBP)

Consider a simplified Iris Recognition example using a variation of Local Binary Pattern (LBP) computed on a small grayscale iris image patch.

Background: Local Binary Pattern (LBP)

- LBP describes texture by comparing a center pixel with its neighbors.
- For each neighbor, assign 1 if pixel ≥ center, else 0.
- The binary pattern forms a number representing local texture.

Problem Setup

Consider a **3×3 iris image patch** (pixel values from 0 to 255):

$$I = \begin{bmatrix} 120 & 115 & 130 \\ 110 & \mathbf{125} & 135 \\ 105 & 100 & 115 \end{bmatrix}$$

- The center pixel is **125** (at position (1,1)).

Iris Recognition Variation of Local Binary Pattern (LBP)

Step 1: Compare neighbors with center pixel

Neighbors in clockwise order starting from top-left:

Position	Value	Comparison to center (125)	Binary bit
Top-left (0,0)	120	120 < 125	0
Top (0,1)	115	115 < 125	0
Top-right(0,2)	130	130 ≥ 125	1
Right (1,2)	135	135 ≥ 125	1
Bottom-right(2,2)	115	115 < 125	0
Bottom (2,1)	100	100 < 125	0
Bottom-left(2,0)	105	105 < 125	0
Left (1,0)	110	110 < 125	0

Iris Recognition Variation of Local Binary Pattern (LBP)

Step 2: Form the LBP binary pattern

Clockwise from top-left neighbor:

0 0 1 1 0 0 0 0

Step 3: Convert binary to decimal

$$(0 \times 2^7) + (0 \times 2^6) + (1 \times 2^5) + (1 \times 2^4) + (0 \times 2^3) + (0 \times 2^2) + (0 \times 2^1) + (0 \times 2^0) = 48$$

Step 4: Use LBP feature for iris recognition

- The decimal **48** is the LBP code for this patch.
- Repeat this for the whole iris image to form a feature histogram.
- Compare histograms of different irises for recognition.

Iris Recognition Variation of Local Binary Pattern (LBP)

Summary:

Neighbor Position	Pixel Value	≥ Center?	Binary Bit
Top-left	120	No	0
Top	115	No	0
Top-right	130	Yes	1
Right	135	Yes	1
Bottom-right	115	No	0
Bottom	100	No	0
Bottom-left	105	No	0
Left	110	No	0
LBP code (decimal)			48

Face Recognition using Local Patterns of Gradients (LPG)

Step 4: Form Local Pattern of Gradients binary code

Clockwise from top-left:

0 0 1 1 0 0 0 0

Step 5: Convert binary to decimal

$$(0 \times 2^7) + (0 \times 2^6) + (1 \times 2^5) + (1 \times 2^4) + (0 \times 2^3) + (0 \times 2^2) + (0 \times 2^1) + (0 \times 2^0) = 48$$

Neighbor	Grad Mag	Bit
Top-left	100	0
Top	150	0
Top-right	180	1
Right	200	1
Bottom-right	160	0
Bottom	140	0
Bottom-left	130	0
Left	110	0

Summary:

LPG code: 48

Use in Face Recognition

- Compute LPG codes for many patches over face image.
- Form histogram of LPG codes as face descriptor.
- Compare histograms to recognize faces.

Vehicle Count Prediction from Sensor Data

A Vehicle Count Prediction from Sensor Data using a simple threshold-based rule and optionally linear regression.

Suppose a ground sensor records the number of "activations" per second over a 10-second period:

Sensor counts=[0,1,3,5,4,5,3,1,0,0]

- Each activation is a car wheel.
- Assume each vehicle has 2 axles (→ 2 pulses per car).

Step 1: Total Pulse Count

Total activations=0+1+3+5+4+5+3+1+0+0=22

Step 2: Estimate Number of Vehicles

Assuming **2 pulses per vehicle** (front + rear axle):

Vehicle count=22/2=11

Predicted Vehicle Count: 11 vehicles

Vehicle Count Prediction from Sensor Data

A Vehicle Count Prediction from Sensor Data using a simple threshold-based rule and optionally linear regression.

Suppose a ground sensor records the number of "activations" per second over a 10-second period:

Sensor counts=[0,1,3,5,4,5,3,1,0,0]

- Each activation is a car wheel.
- Assume each vehicle has 2 axles (→ 2 pulses per car).

Pulses	Vehicles
10	5
12	6
20	10
24	12

Vehicles=Pulses/2

linear model: $y=0.5x$

$y=0.5\times22=11$

Object Count in Image

Problem: Count the number of distinct objects in a binary image.

Input binary image (5×5 pixels)

Suppose **1** represents object pixels and 0 represents background:

	0	**0**	**0**	**1**	**1**	**0**
	0	0	0	1	1	0
$I =$	0	0	0	0	0	0
	1	1	0	0	0	1
	1	1	0	0	1	1

Identify connected components (objects)

Use **4-connectivity** (up, down, left, right neighbors):

- Label each connected group of 1's with a unique label.

Object Count in Image

	0	0	0	1	1	0
	0	0	0	1	1	0
$I =$	0	0	0	0	0	0
	1	1	0	0	0	1
	1	1	0	0	1	1

Use **4-connectivity**
Start scanning from top-left:

Pixel Position	Pixel Value	Action	Label Assigned
(1,3)	1	New object	1
(1,4)	1	Connected to (1,3)	1
(2,3)	1	Connected to above	1
(2,4)	1	Connected to above	1
(4,1)	1	New object	2
(4,2)	1	Connected to (4,1)	2
(4,5)	1	New object	3
(5,1)	1	Connected to (4,1)	2
(5,2)	1	Connected to (4,2)	2
(5,4)	1	Connected to (4,5)	3
(5,5)	1	Connected to (4,5)	3

Object Count in Image

Pixel Position	Pixel Value	Action	Label Assigned
(1,3)	1	New object	1
(1,4)	1	Connected to (1,3)	1
(2,3)	1	Connected to above	1
(2,4)	1	Connected to above	1
(4,1)	1	New object	2
(4,2)	1	Connected to (4,1)	2
(4,5)	1	New object	3
(5,1)	1	Connected to (4,1)	2
(5,2)	1	Connected to (4,2)	2
(5,4)	1	Connected to (4,5)	3
(5,5)	1	Connected to (4,5)	3

Labeled image

	0	**0**	**0**	**1**	**1**	**0**
	0	0	0	1	1	0
$L =$	0	0	0	0	0	0
	2	2	0	0	0	3
	2	2	0	0	3	3

Count objects

- Labels found: 1, 2, 3
- **Number of objects = 3**

Thank You

Lecture 5.3

Image Segmentation

Otsu's Thresholding Algorithm

Instructor Name:

Prof. Dr. Jahangir Alam

Dept. of CSE, BAUST

Otsu's Thresholding Algorithm

- Automatically finds the **optimal threshold** to separate foreground and background by **maximizing between-class variance** or equivalently minimizing intra-class variance.

- Works on image histogram (gray level distribution).

Interpretation

- When between-class variance is maximum → classes are best separated.
- It's a non-iterative global method (no initial guess needed).

Otsu's Method/ Automatic Global Thresholding

From a video frame, detect the license plate region and recognize the characters by first binarizing the image using **Otsu's thresholding**.

Example Applications

- Brain tumor segmentation in MRI
- Bone and soft-tissue separation in X-ray or CT
- Cell segmentation in microscopy

Otsu's Thresholding Algorithm

Advantages

- Fully automatic (no parameters or iterations)– no need to manually pick a threshold.
- Fast and simple to implement
- Works well for **bimodal histograms**
- Simple and fast.
- Works well for images with two distinct intensity peaks.

Limitations

- Not ideal for multi-modal histograms.
- Sensitive to unequal class sizes.
- Assumes bimodality (foreground and background intensities are distinct).

Otsu's Thresholding Algorithm

1. Compute histogram and probabilities, P_i: Compute the normalized histogram of the input image. Denote the components of the histogram by

$$p_i, i = 0, 1, 2, \ldots, L - 1.$$

$$p_i = \frac{n_i}{N} \Rightarrow \sum_{i=0}^{L-1} p_i = 1$$

2. Choose a threshold T

Split the pixels into two classes:

- $C_1 = [0,1,\ldots,T] \rightarrow$ background
- $C_2 = [T+1,\ldots,L-1] \rightarrow$ foreground

Otsu's Thresholding Algorithm

1. For every possible threshold T:

- Compute class probabilities:

$$w_1(T) = \sum_{i=0}^{T} p_i \ , w_2(T) = \sum_{i=T+1}^{L-1} p_i = 1 - w_1(T)$$

4. Compute class means

$$\mu_1(T) = \frac{\sum_{i=0}^{T} i\, p_i}{w_1(T)}, \qquad \mu_2(T) = \frac{\sum_{i=T+1}^{L-1} i\, p_i}{w_1(T)}$$

5. Compute cumulative means, $m(k)$, for $k = 0, 1, 2, \ldots, L-1$.

$$m(k) = \sum_{i=0}^{k} i\, p_i$$

Otsu's Thresholding Algorithm

6. Compute the global intensity mean, m_G.

$$\mu_T = m_G = \sum_{i=0}^{L-1} i\, p_i$$

7. Compute the between-class variance, $\sigma_B^2(k)$,
$for\ k = 0, 1, 2, \ldots, L - 1.$

$$\sigma_B^2(T) = w_1(T)[\mu_1(T) - \mu_T]^2 + w_2(T)[\mu_2(T) - \mu_T]^2$$

Or,

$$\sigma_B^2(T) = w_1(T)\, w_2(T)[\mu_1(T) - \mu_2(T)]^2$$

Otsu's Thresholding Algorithm

8. Obtain the Otsu threshold k for $\sigma_B^2(k)$ is maxi-mum. If the maximum is not unique, obtain k by averaging.

9. Obtain the separability measure, η.

$$\eta(k) = \frac{\sigma_B^2(k)}{\sigma_G^2}$$

10. Find optimal threshold

$$T^* = arg \max_T \sigma_B^2(T)$$

That T^* gives the maximum separation between the two classes.

Otsu's Thresholding Algorithm

Consider a small grayscale image with pixels having intensities from 0 to 3:

$$I = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 1 & 2 & 2 & 3 \\ 3 & 3 & 3 & 3 \end{bmatrix}$$

Step 1: Calculate histogram and probability of each intensity level: Count of each intensity:

Intensity (i)	Count, h_i	Probability, $pi = \frac{hi}{N}$
0	2	2/11=0.182
1	3	3/11=0.273
2	2	2/11=0.182
3	4	4/11=0.364
	$\sum h_i = 11$	$\sum p_i = 1$

Otsu's Thresholding Algorithm

Step 2: Test thresholds $t = 0, 1, 2$ (possible thresholds between intensities)

For each threshold *t*:

- Class 1: $intensities \leq t$
- Class 2: $intensities > t$

Calculate for each $\boldsymbol{t} \in \{0, 1, 2, 3\}$:

- Class probabilities:

	Class 1	Class 2
Probabilities	$\omega_1(t) = \sum_{i=0}^{t} p_i$	$\omega_2(t) = \sum_{i=t+1}^{4} p_i = 1 - \omega_1(t)$
Mean	$\mu_1(t) = \frac{\sum_{i=0}^{t} i\, p_i}{\omega_1(t)}$	$\mu_2(t) = \frac{\sum_{i=t+1}^{4} i\, p_i}{\omega_1(t)}$

Otsu's Thresholding Algorithm

- **Global mean intensity,**

$$\mu_T = \sum_{i=0}^{n} i\, p_i = \sum_{i=0}^{4} i\, p_i$$

$$\begin{bmatrix} 0 & 0 & 1 & 1 \\ 1 & 2 & 2 & 3 \\ 3 & 3 & 3 & 3 \end{bmatrix}$$

0	0.182
1	0.273
2	0.182
3	0.364

- **Between-class variance,**

$$\sigma_B^2(t) = \omega_1(\mu_1 - \mu_T)^2 + \omega_2(\mu_2 - \mu_T)^2$$

$$\text{Or, } \sigma_B^2(t) = \omega_1 \times \omega_2 \times (\mu_1 - \mu_2)^2$$

Calculate μ_T:

$$\mu_T = \sum_i i\, p_i = 0 \times 0.182 + 1 \times 0.273 + 2 \times 0.182 + 3 \times 0.364$$

$$= 1.728$$

Otsu's Thresholding Algorithm

0	0.182
1	0.273
2	0.182
3	0.364

Threshold $t = 0$:

- $\omega_1(t) = p_0 = 0.182$
- $\omega_2(t) = 1 - 0.182 = 0.818$
- $\mu_1(t) = \frac{i \times \omega_1(t)}{\omega_1(t)} = \frac{0 \times 0.182}{0.182} = 0$
- $\mu_2(t) = \frac{\sum_i i\, p_i}{\omega_2(t)} = \frac{1 \times 0.273 + 2 \times 0.182 + 3 \times 0.364}{0.818} = 2.112$

$$\sigma_B^2(t) = \omega_1 \times \omega_2 \times (\mu_1 - \mu_2)^2$$

$$= 0.182 \times 0.818 \times (0 - 2.112)^2 \approx 0.66$$

Otsu's Thresholding Algorithm

Threshold $t = 1$:

0	0.182
1	0.273
2	0.182
3	0.364

- $\omega_1(t) = p_0 + p_1 = 0.182 + 0.273 = 0.455$
- $\omega_2(t) = 1 - 0.455 = 0.545$
- $\mu_1(t) = \frac{0\times0.182+1\times0.273}{0.455} = 0.6$
- $\mu_2(t) = \frac{2\times0.182+3\times0.364}{0.545} = 2.67$

$$\sigma_B^2(t) = \omega_1 \times \omega_2 \times (\mu_1 - \mu_2)^2$$

$$= 0.455 \times 0.545 \times (0.6 - 2.67)^2 \approx 1.06$$

Otsu's Thresholding Algorithm

Threshold $t = 2$:

- $\omega_1(t) = p_0 + p_1 + p_2 = 0.182 + 0.273 + 0.182 = 0.636$
- $\omega_2(t) = 1 - 0.636 = 0.364$
- $\mu_1(t) = \frac{0\times0.182+1\times0.273+2\times0.182}{0.636} = 0.983$
- $\mu_2(t) = \frac{3\times0.364}{0.364} = 3.0$

0	0.182
1	0.273
2	0.182
3	0.364

$$\sigma_B^2(t) = \omega_1 \times \omega_2 \times (\mu_1 - \mu_2)^2$$

$$= 0.636 \times 0.364 \times (0.983 - 3.0)^2 \approx 0.94$$

Otsu's Thresholding Algorithm

Threshold $t = 3$:

- $\omega_1(t) = p_0 + p_1 + p_2 + p_3 = 0.182 + 0.273 + 0.182 + 0.364 = 1$
- $\omega_2(t) = 1 - 1 = 0$
- $\mu_1(t) = 0$
- $\mu_2(t) = 0$
- $\sigma_B^2(t) = 0$

Step 3: Choose threshold with max between-class variance

Result: Optimal threshold *t=1*.

Threshold	$\sigma_B^2(t)$
0	0.66
1	1.06
2	0.94
3	0

Otsu's Thresholding Algorithm

Step 5: Apply threshold on image pixels

- Pixels < 1 → background (0)
- Pixels ≥ 1 → foreground (1)

$$\begin{bmatrix} 0 & 0 & 1 & 1 \\ 1 & 2 & 2 & 3 \\ 3 & 3 & 3 & 3 \end{bmatrix}$$

Segmented image:

$$I_{segmented} = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Otsu's Thresholding Algorithm

Detecting and Recognizing a Car License Plate from video in real time using Otsu's Algorithm (for binarization), illustrating the key parts clearly.

Step 1: Video Frame Input (Simplified 5×5 grayscale image)
Grayscale pixel intensities of a cropped license plate region:

120	125	130	135	140
122	128	255	125	142
118	255	255	130	135
115	120	125	128	130
110	112	115	120	125

- Brighter pixels (around 255) likely represent characters (white text)

- Darker pixels represent plate background (gray tones)

Otsu's Thresholding Algorithm

Step 2: Apply Otsu's Algorithm to Binarize the Image

a) Find pixel intensity histogram (simplified):

Intensity	Frequency
110-120	8
125-130	7
135-142	4
255	4



120	125	**130**	135	**140**
122	128	255	125	142
118	255	255	130	135
115	120	125	128	130
110	112	115	120	125

b) Compute between-class variance for possible thresholds:

•Threshold candidates: 120, 130, 140, 200

Calculate weighted variances for these thresholds (for simplicity, pick the best threshold based on intuition):

•Threshold = 140 separates background (<=140) and characters (>140)

Otsu's Thresholding Algorithm

c) Binarize image at threshold T= 140:

Set pixels > 140 → 1 (white), else 0 (black)

0	0	0	0	0
0	0	**1**	0	**1**
0	**1**	**1**	0	0
0	0	0	0	0
0	0	0	0	0

Step 3: Character Segmentation

- Identify connected components (groups of adjacent white pixels).

Here, two groups of white pixels correspond to different characters.

Step 4: Character Recognition (Simple template matching)

- Compare segmented characters' shapes with known character templates (e.g., numbers/letters).
- *For example*:
  - Component 1 shape matches '1'
  - Component 2 shape matches '0'

Step 5: Output

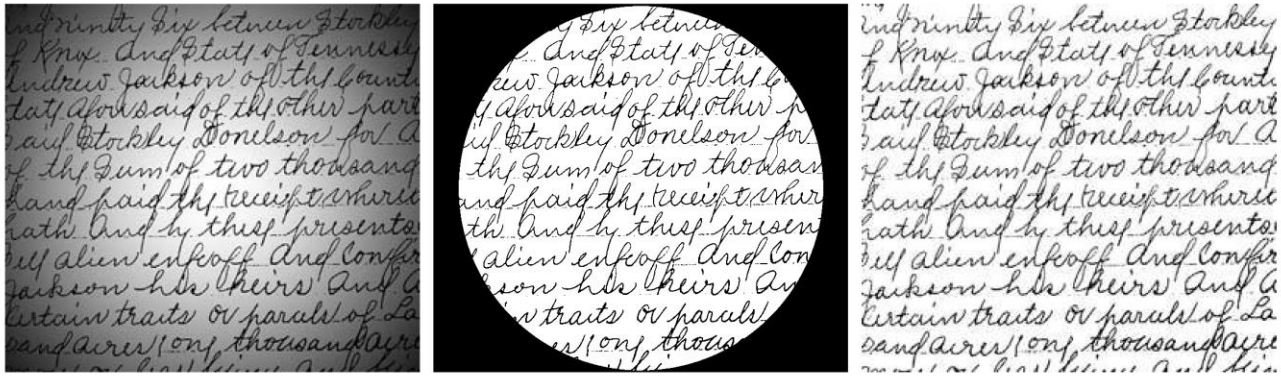
Recognized plate number segment: **"10"**

Otsu's Thresholding Algorithm

Summary:

Step	Description	Example Result
Input Frame	Cropped grayscale plate region	5×5 grayscale image
Otsu's Thresholding	Compute best threshold and binarize	Threshold = 140
Binarized Image	Convert grayscale to binary image	Binary 0/1 matrix
Segmentation	Find connected white regions	Two-character components
Recognition	Match to known character templates	Characters '1' and '0'
Output	License plate characters recognized	"10"

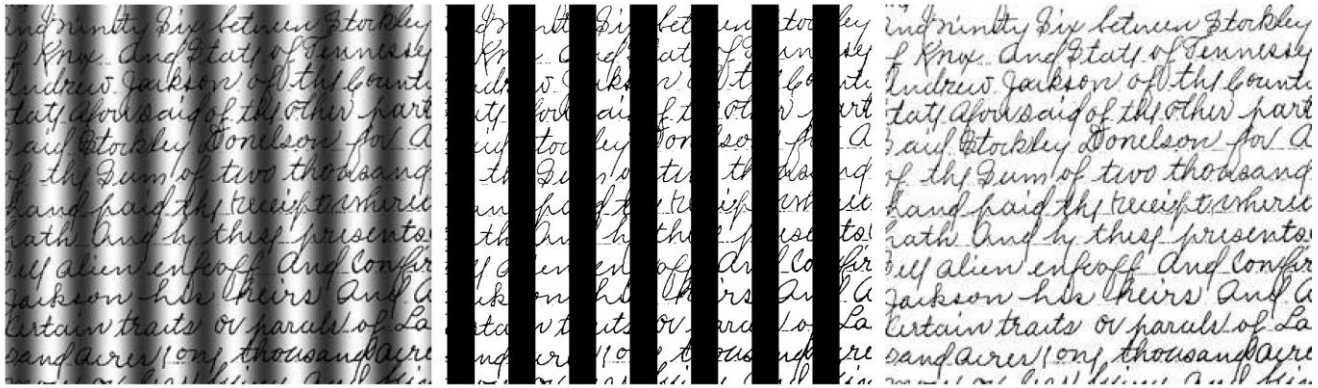
Optimum Global Thresholding Using Otsu's Method



a b c

FIGURE 10.49 (a) Text image corrupted by spot shading. (b) Result of global thresholding using Otsu's method. (c) Result of local thresholding using moving averages.

Optimum Global Thresholding Using Otsu's Method



a b c

FIGURE 10.50 (a) Text image corrupted by sinusoidal shading. (b) Result of global thresholding using Otsu's method. (c) Result of local thresholding using moving averages.

Thank You

Lecture 5.4

Image Segmentation

Point, Line, Detection

Instructor Name:
Professor Dr. Jahangir Alam
Professor, Dept. of CSE, BAUST

Point Detection of Image

$$I(i,j) = \begin{bmatrix} 10 & 10 & 10 \\ 10 & 50 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

Notice: center pixel = 50 is much brighter → potential point.

Point Detection Mask (Laplacian of Gaussian or simplified Laplacian):

$$M_{point}(i,j) = \begin{bmatrix} -1 & -1 & -1 \\ -1 & \mathbf{8} & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Convolution: sum of element-wise multiplication

$$R = \sum M_{point}(i,j) \times I(i,j)$$

$$= (-1)(10) + (-1)(10) + (-1)(10) + (-1)(10) + \mathbf{8(50)}$$
$$+ (-1)(10) + (-1)(10) + (-1)(10) + (-1)(10) = 320$$

High positive response → **point detected at center**.

Detection of Line

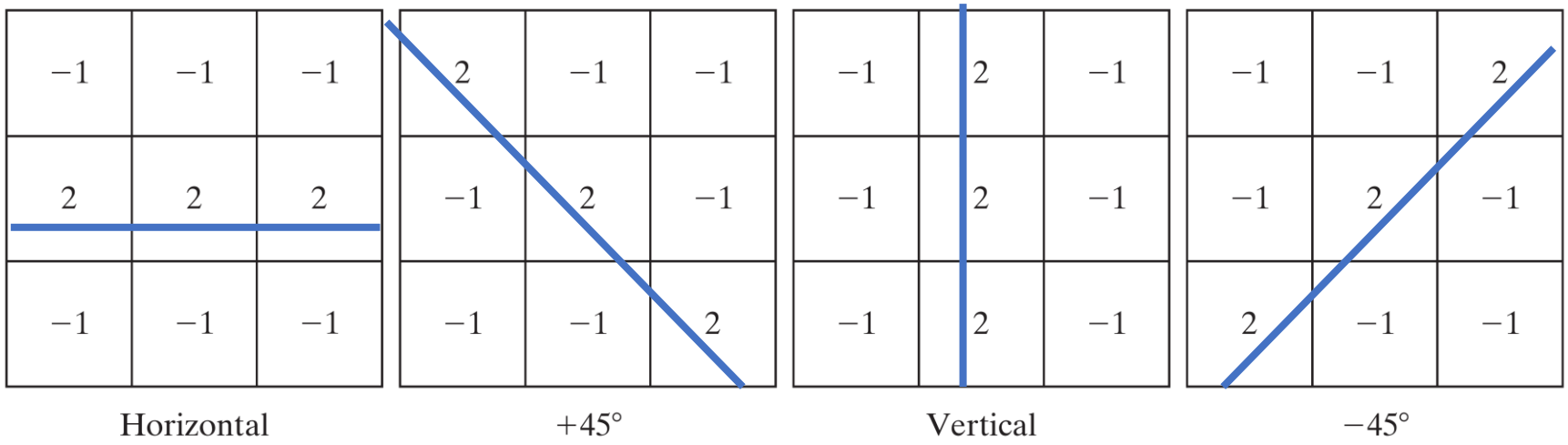


FIGURE 10.6 Line detection masks. Angles are with respect to the axis system in Fig. 2.18(b).

Line Detection of Image

Line Detection Example

Image Patch (3×3):

$$I(i,j) = \begin{bmatrix} 0 & 0 & 0 \\ 255 & 255 & 255 \\ 0 & 0 & 0 \end{bmatrix}$$

A horizontal bright line in middle row.

Line Detection Mask (horizontal line):

$$M_{line}(i,j) = \begin{bmatrix} -1 & -1 & -1 \\ 2 & 2 & 2 \\ -1 & -1 & -1 \end{bmatrix}$$

Convolution:

$$R = \sum M_{line}(i,j) \times I(i,j)$$

$$= (-1)(0) + (-1)(0) + (-1)(0) + 2(255) + 2(255) + 2(255) + (-1)(0) + (-1)(0) + (-1)(0) = 0 + 1530 + 0 = 1530$$

High positive → **horizontal line detected**.

Line Detection using Hough Transform

The **Hough Transform** is a powerful technique in **image processing** used to detect geometric shapes—especially **lines, circles, and curves**—even when the image is noisy or incomplete.

Instead of detecting shapes directly in the image space, the Hough Transform works in a **parameter space**.

Problem Setup

You have a small 5×5 binary image (1 = edge pixel, 0 = background):

$$f = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

This clearly shows a vertical line in the 3rd column.

Line Detection using Hough Transform

The **Hough Transform** is a powerful technique in **image processing** used to detect geometric shapes—especially **lines, circles, and curves**—even when the image is noisy or incomplete.

Instead of detecting shapes directly in the image space, the Hough Transform works in a **parameter space**.

Each point in the **image votes** for all possible shapes that could pass through it.

The shape with the highest votes is detected.

Line Detection using Hough Transform

A straight line is usually written as: $y = mx + c$

But this form fails for vertical lines. So we use the **polar form**:

$$\rho = x\cos\theta + y\sin\theta$$

Where:

- ρ= distance from origin to the line
- θ= angle of the normal to the line
- (x, y) =pixel coordinates

Types of Hough Transform

1. Standard Hough Transform

- Detects infinite lines.

2. Probabilistic Hough Transform

- Faster, detects line segments instead of full lines.

3. Circular Hough Transform

- Detects circles using parameters:

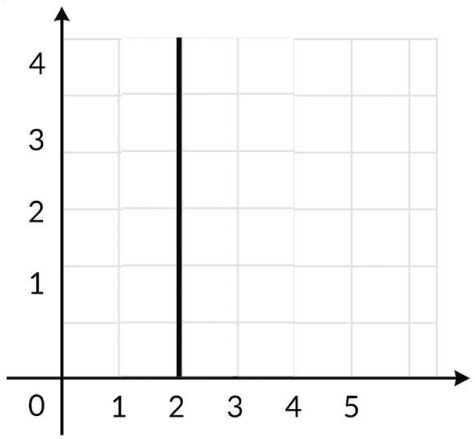
$$\left(x - a\right)^2 + \left(y - b\right)^2 = r^2$$

Line Detection using Hough Transform

Step 1: Edge Points Coordinates

List coordinates of edge pixels $(x, y) = (col, row)$:

$$(x, y) = \{(2,0), (2,1), (2,2), (2,3), (2,4)\}$$



Step 2: Parameterize Lines in (ρ, θ) space

- $\rho = x\,cos\theta + y\,sin\theta$
- $\theta \in [0^o, 180^o)$— we'll check a few discrete angles: 0°, 45°, 90°, 135°

Line Detection using Hough Transform

Step 3: Compute ρ for each edge point and each θ

Assuming pixel coords: *x=col, y=row*

$P(x,y)$	$\rho = x\cos\theta + y\sin\theta$			
	$\theta = 0^o$	$\theta = 45^o$	$\theta = 90^o$	$\theta = 135^o$
P1=(2,0)	$\rho = 2 \times cos0^o + 0 \times sin0^o = 2$	$\rho = 2 \times cos45^o + 0 \times sin45^o$=1	$\rho = 2 \times cos90^o + 0 \times sin90^o$=0	$\rho = 2 \times cos135^o + 0 \times sin135^o$= --1
P2=(2,1)	2	2	1	--1
P3=(2,2)	2	3	2	0
P4=(2,3)	2	4	3	1
P5=(2,4)	2	4	4	1
Accumulator:	$\rho = 2 \rightarrow 5\ votes$	$\rho = 1 \rightarrow 1\ votes$ $\rho = 2 \rightarrow 1$ $\rho = 3 \rightarrow 1$ $\rho = 4 \rightarrow 2$	$\rho = 0 \rightarrow 1\ votes$ $\rho = 1 \rightarrow 1$ $\rho = 2 \rightarrow 1$ $\rho = 3 \rightarrow 1$ $\rho = 4 \rightarrow 1$	$\rho = -1 \rightarrow 2\ votes$ $\rho = 0 \rightarrow 1$ $\rho = 1 \rightarrow 2$

Line Detection using Hough Transform

Step 4: Accumulate Votes in Hough Space

- For $\boldsymbol{\theta} = 0^o$, all points vote at ρ.
- For other $\boldsymbol{\theta}$, votes spread over different ρ.
- Collecting maxima from each θ:
- For $\theta = 0^o$, maxima found with $\rho = 2 \rightarrow 5\ votes$
- Global peak = ($\theta = 0^o$, $\rho = 2$) with 5 votes.
- That strongly indicates a vertical line at $x = 2$.

Step 5: Detect Peak

•Peak at $\rho=2$, $\boldsymbol{\theta=0o}$with 5 votes → detected line.

$$\rho = x\cos\theta + y\sin\theta \Rightarrow 2 = x(1) + y(0) \Rightarrow x = 2$$

•Corresponds to vertical line $x=2$ (3rd column).

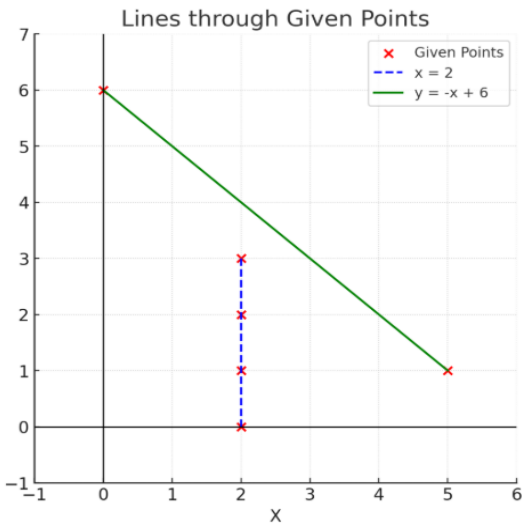
Line Detection using Hough Transform

Summary:

Step	Description
Edge detection	Find edge pixels in image
Parameterization	Represent lines as ρ, θ pairs
Voting	Each edge pixel votes for possible lines
Peak detection	Find highest votes to identify line

Line Detection using Hough Transformation

P1=(2,0), P2=(2,1), P3=(2,2)
P4=(2,3) ← these four lie on the vertical line $x = 2$
P5=(0,6)← random noise point
P6=(5,1) ← another random point
We want Hough to find the dominant line.



$P(x,y)$	$\rho = x\cos\theta + y\sin\theta$			
	$\theta = 0^o$	$\theta = 45^o$	$\theta = 90^o$	$\theta = 135^o$
P1=(2,0)	$\rho = 2 \times cos0^o + 0 \times sin0^o = 2$	$\rho = 2 \times cos45^o + 0 \times sin45^o$=1	$\rho = 2 \times cos90^o + 0 \times sin90^o$=0	$\rho = 2 \times cos135^o + 0 \times sin135^o$=-1
P2=(2,1)	2	2	1	-1
P3=(2,2)	2	3	2	0
P4=(2,3)	2	4	3	1
P5=(0,6)	0	4	6	4
P6=(5,1)	5	4	1	-3

Line Detection using Hough Transformation

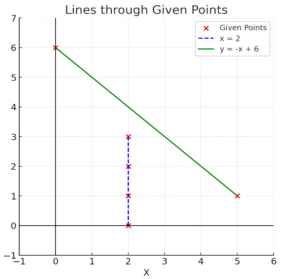
$P(x,y)$	$\rho = x\cos\theta + y\sin\theta$			
	$\theta = 0^o$	$\theta = 45^o$	$\theta = 90^o$	$\theta = 135^o$
P1=(2,0)	$\rho = 2 \times cos0^o + 0 \times sin0^o = 2$	$\rho = 2 \times cos45^o + 0 \times sin45^o$=1	$\rho = 2 \times cos90^o + 0 \times sin90^o$=0	$\rho = 2 \times cos135^o + 0 \times sin135^o$=-1
P2=(2,1)	2	2	1	-1
P3=(2,2)	2	3	2	0
P4=(2,3)	2	4	3	1
P5=(0,6)	0	4	6	4
P6=(5,1)	5	4	1	-3
Accumulator:	$\rho = 2 \rightarrow 4\ votes$ $\rho = 0 \rightarrow 1$ $\rho = 5 \rightarrow 1$	$\rho = 1 \rightarrow 1\ votes$ $\rho = 2 \rightarrow 1$ $\rho = 3 \rightarrow 1$ $\rho = 4 \rightarrow 3$	$\rho = 0 \rightarrow 1\ votes$ $\rho = 1 \rightarrow 2$ $\rho = 2 \rightarrow 1$ $\rho = 3 \rightarrow 1$ $\rho = 6 \rightarrow 1$	$\rho = -1 \rightarrow 2\ votes$ $\rho = 0 \rightarrow 1$ $\rho = 1 \rightarrow 1$ $\rho = 4 \rightarrow 1$ $\rho = -3 \rightarrow 1$

Collecting maxima from each θ:

For $\theta = 0^o$, maxima found with $\rho = 2 \rightarrow 4\ votes$

Global peak = ($\theta = 0^o$, $\rho = 2$) with 4 votes.

That strongly indicates a vertical line at $x = 2$.

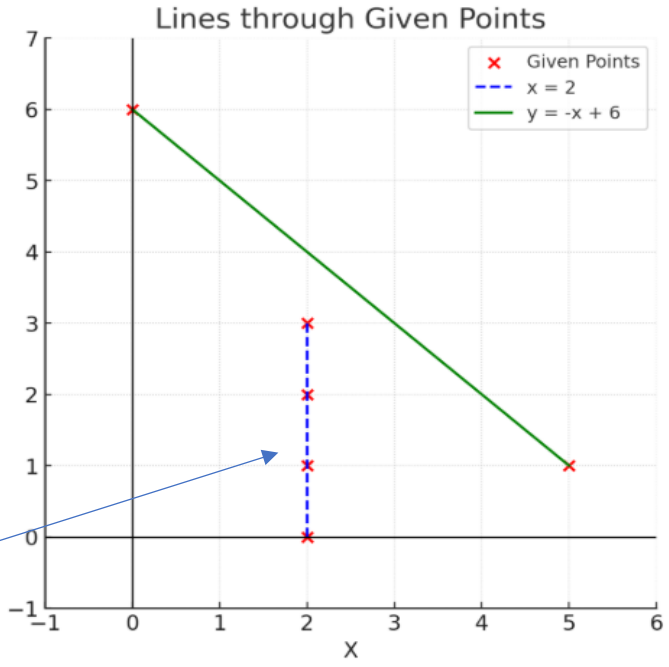


Line Detection using Hough Transformation

With $\theta = 0^o, \rho = 2$

$$\rho = x cos\theta + y sin\theta$$
$$\Rightarrow 2 = x(1) + y(0)$$
$$\Rightarrow x = 2$$

So, the detected line is $x = 2$ — exactly the vertical line passing through P1, P2, P3, P4.



With $\theta = 135^o, \rho = -6$

$$-6 = -x - y \Rightarrow y = -x + 6$$

Thank You

Lecture 5.5

Image Segmentation

Edges Detection

Instructor Name:
Professor Dr. Jahangir Alam
Professor, Dept. of CSE, BAUST

Edge-Based Segmentation

Detect object boundaries using pixel intensity gradients. Uses image gradients to find object boundaries.

- Common operators: **Sobel**, **Prewitt**, **Canny**.

- Works well when edges are clear, but sensitive to noise.

$$G(x,y) = \sqrt{(f_x)^2+(f_y)^2}$$

- f_x, f_y: derivatives/gradients in x, y directions.

Example:

✓ Used in X-ray bone edges, vessel outlines, etc.

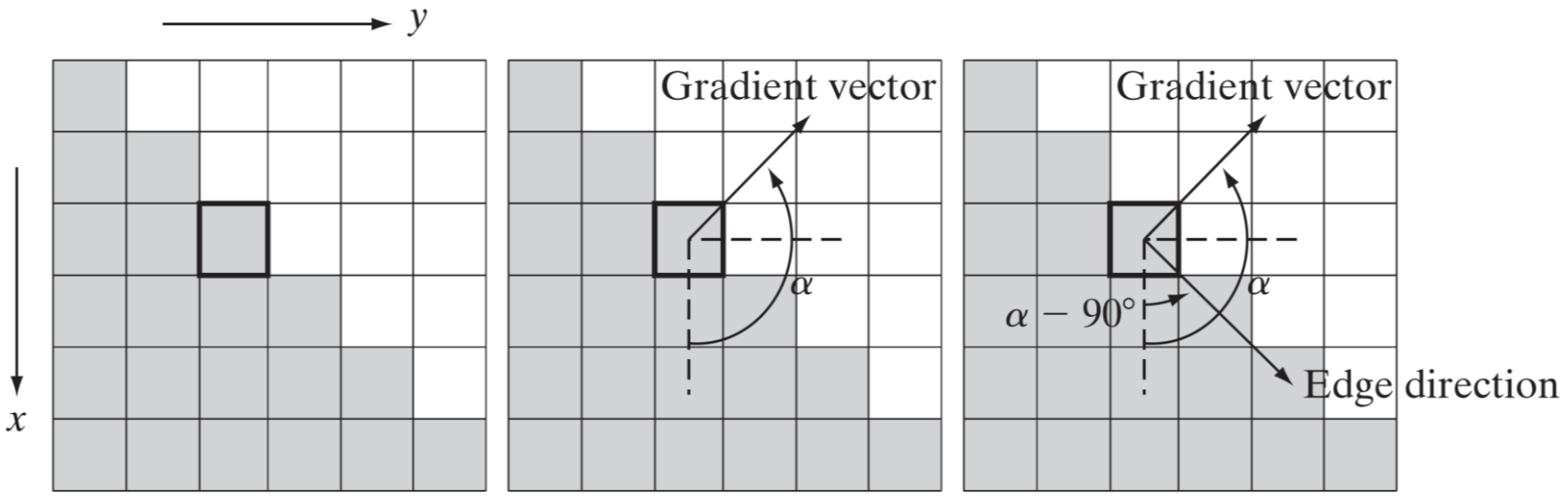
✓ Finding outlines of organs or blood vessels.

Detection of Edges

Step 4: Compute Gradient Direction (Optional)

$$\theta = tan^{-1}\left(\frac{G_y}{G_x}\right)$$

This gives the **orientation of the edge**.



Edge Detection and Linking

Objective:

1.Detect edges using gradients (like Sobel).

2.Apply Non-Maximum Suppression (NMS) to thin out edges.

3.Apply Hysteresis Thresholding to link weak edges to strong edges if they're connected.

Step 1: Small Grayscale Image (5×5)

Assume pixel intensity matrix I:

Center is a bright circular "blob" — like a simple object.

	10	10	10	10	10
	10	100	100	100	10
$I =$	10	150	255	150	10
	10	100	150	100	10
	10	10	10	10	10

Edge Detection and Linking

Step 2: Compute Gradient Magnitude

We apply Sobel Operator to compute gradient magnitude (G) — simplified here:

We'll directly list a hypothetical gradient magnitude output:

	0	0	0	0	0
	0	70	90	70	0
$G =$	0	90	0	90	0
	0	70	90	70	0
	0	0	0	0	0

Interpretation:

•The highest gradients (edges) surround the center (which is flat, hence G = 0 at (2,2)).

Edge Detection and Linking

Step 3: Non-Maximum Suppression (NMS)

•NMS thins edges by keeping only the *local maximum* along the direction of the gradient.

•For simplicity, let's assume the direction is outward from the center.

•Only keep values if they're greater than neighbors in the same gradient direction.

Let's simulate a result: After NMS,

	0	0	0	0	0
	0	0	90	0	0
$I =$	0	90	0	90	0
	0	0	90	0	0
	0	0	0	0	0

Edge Detection and Linking

Step 4: Hysteresis Thresholding (Edge Linking)

Set:

- High threshold = 80
- Low threshold = 60

Rules:

- If G >= 80 → Strong edge (keep)
- If 60 ≤ G < 80 → Weak edge: keep only if connected to strong edge
- Else → discard

From above NMS result:

- Pixels with 90 → strong edge
- Any weak edge (70) not directly connected to strong edge → removed

The final edge map:

	0	0	0	0	0
	0	0	1	0	0
$I =$	0	1	0	1	0
	0	0	1	0	0
	0	0	0	0	0

Edge Detection and Linking

Applications: Edge linking is critical in,

- Canny edge detection
- Shape tracing
- Contour detection for OCR, face detection

Summary:

Step	Output
Gradient Magnitude	High at edges
Non-Max Suppression	Thins edges
Hysteresis Thresholding	Links edges into complete form

Various Edge Detection Algorithms & Applications

Algorithm	Kernel Size	Noise Sensitivity	Edge Thickness	Use Case
Sobel	3×3	Moderate	Thick	General edge detection
Prewitt	3×3	Moderate	Thick	Simpler, faster processing
Roberts	2×2	High	Thin	High-res, real-time
Canny	Variable	Low (good noise suppression)	Thin	Best overall performance
Laplace	Variable	High	Thin	Best overall performance
LoG	Variable	Moderate	Thin	Edge + blob detection
DoG	Variable	Low	Zero-crossing	Edge + blob detection

Sobel Edge Detection Algorithm

Most fundamental and widely used edge detection techniques.

To detect edges (areas of rapid intensity change) by computing gradients in the x and y directions using the Sobel operator, and

Combining them to get the gradient magnitude.

Advantages

- ✓ Simple and fast (uses small convolution kernels)
- ✓ Detects both vertical and horizontal edges
- ✓ Smooths noise slightly (built-in averaging effect)

Sobel Edge Detection Algorithm

Applications

Application	Use
Image Segmentation	Find object boundaries
Medical Imaging	Detect bone or tissue edges
Computer Vision	Object detection, tracking
Document Analysis	Text and line extraction

Step 1: Convert to Grayscale

If the input is a color image, convert it to **grayscale**, since edges are intensity-based.

$$I(x, y) = 0.299R + 0.587G + 0.114B$$

Sobel Edge Detection Algorithm

Step 2: Apply Sobel Filters

Two 3×3 convolution kernels are used by detecting horizontal changes (Edge) (Gx) and vertical changes (Edge) (Gy).

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Each kernel is **convolved** with the image.

Step 3: Compute Gradient Magnitude

$G = \sqrt{G_x^2 + G_y^2}$ Or approximate with: $G = |Gx| + |Gy|$

This gives the **edge strength** at each pixel — large values → strong edges.

Sobel Edge Detection Algorithm

Step 4: Compute Gradient Direction (Optional)

$$\theta = tan^{-1}\left(\frac{G_y}{G_x}\right)$$

This gives the **orientation of the edge**.

Step 5: Thresholding

Choose a threshold T to keep only significant edges:

$$E(x,y) = \begin{cases} 1 & |\, G(x,y)\,| \geq T \\ 0 & |\, G(x,y)\,| < T \end{cases}$$

This gives a **binary edge map**.

Sobel Edge Detection Algorithm Steps

Input Image

↓

Convert to Grayscale

↓

Gradient Calculation

(Compute edge strength & direction using Sobel Gradient (Gx, Gy))

↓

Edge Magnitude & Direction

↓

Threshold |G| → Binary Edge Map

↓

Output: Edge Detected Image

Bone Fracture Detection

Problem Setup: You have a small 5×5 grayscale X-ray image patch of a bone area, I:

- Normal bone tissue intensities: ~100-150
- Fracture region (dark line) pixels: low intensities around 35-45.

100	102	104	103	100
99	150	40	148	101
98	152	35	147	102
100	151	45	149	103
101	103	105	104	102

Step 1: Thresholding to segment fracture

Set a threshold T=80 to isolate dark fracture pixels, I_{th}:

0	0	0	0	0
0	0	**1**	0	0
0	0	**1**	0	0
0	0	**1**	0	0
0	0	0	0	0

Where 1 indicates potential fracture.

Bone Fracture Detection

Step 2: Morphological processing (optional)

- Use **dilation** to connect fragmented pixels.
- Here, the fracture pixels form a clear vertical line.

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Step 3: Edge detection (Sobel operator)

Calculate approx. horizontal & vertical gradients around suspected fracture. Vertical gradient highlights vertical edges (**fracture line**).

Example for center pixel (2,2):

$$G_x$$
$$= (-1) \times I(1,1) + 0 \times I(1,2) + 1 \times I(1,3) + (-2) \times I(2,1) + 0 \times I(2,2) + 2 \times I(2,3) + (-1) \times I(3,1) + 0 \times I(3,2) + 1 \times I(3,3)$$
$$= (-1) \times 150 + 0 \times 40 + 1 \times 148 + (-2) \times 152 + 0 \times 35 + 2 \times 147 + (-1) \times 151 + 0 \times 45 + 1 \times 149 = -14$$

A strong gradient magnitude here indicates an edge.

Bone Fracture Detection

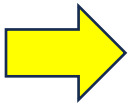
Step 4: Identify fracture line

- Pixels with high gradient magnitude and low intensity indicate fracture edges.
- The vertical line of low-intensity pixels with strong vertical gradients likely corresponds to a fracture.

Step 5: Highlight fracture on original image

Set fracture pixels to maximum intensity (255), $I_{highlighted}$:

100	102	104	103	100
99	150	40	148	101
98	152	35	147	102
100	151	45	149	103
101	103	105	104	102



100	102	104	103	100
99	150	**255**	148	101
98	152	**255**	147	102
100	151	**255**	149	103
101	103	105	104	102

Sobel Edge Detection

$$G = \begin{bmatrix} 100 & 100 & 100 \\ 100 & 50 & 50 \\ 100 & 50 & 50 \end{bmatrix}$$

Sobel Kernels:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Apply these at the **center pixel** (value 50) using convolution.

$$G_x = (-1)(100) + (0)(100) + (1)(100) + (-2)(100) + (0)(50) + (2)(50) + (-1)(100) + (0)(50) + (1)(50) = -150$$

$$G_y = (1)(100) + (2)(100) + (1)(100) + (0)(100) + (0)(50) + (0)(50) + (-1)(100) + (-2)(50) + (-1)(50) = 150$$

Sobel Edge Detection

Calculate Gradient Magnitude

$$G = \sqrt{(-150)^2 + (150)^2} = 212.13$$

Or approximate with:

$$G = |-150| + |150| = 300$$

Final Result:

- Edge strength at the center pixel ≈ **212**
- The center pixel has a **strong edge** ($gradient \approx 212 - 300$), indicating a significant change in intensity — likely a boundary or edge.
- Strong edge detected — confirms change from high (100) to low (50) intensity.

Sobel Edge Detection of Image

$$\begin{bmatrix} 10 & 50 & 50 \\ 10 & \mathbf{50} & 50 \\ 10 & 50 & 50 \end{bmatrix}$$

Sobel Horizontal Edge Mask:

$$G_x = M_{edge}(i,j) = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y = M_{edge}(i,j) = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

$$R_x = \sum M_{point}(i,j) \times I(i,j)$$
$$= (-1)(10) + 0(50) + 1(50) + (-2)(10) + 0(50) + 2(50)$$
$$+ (-1)(10) + 0(50) + 1(50) = 160$$

Positive response → **edge detected horizontally**.

$$R_y = \sum M_{point}(i,j) \times I(i,j)$$
$$= (-1)(10) + (-2)(50) + (-1)(50) + 0(10) + 0(50) + 0(50)$$
$$+ (1)(10) + 2(50) + 1(50) = 0$$

Positive response → **edge detected vertically**.

Sobel Edge Detection of Image

$$\begin{bmatrix} 10 & 50 & 50 \\ 10 & \mathbf{50} & 50 \\ 10 & 50 & 50 \end{bmatrix}$$

Gradient magnitude:

$$|R| = \sqrt{R_x^2 + R_y^2} = \sqrt{(160)^2 + (0)^2} = 160$$

$$\theta = tan^{-1}2(R_y, R_x) = tan^{-1}2(0,160) = 0^o$$

Direction:

Gradient points to $+x$), which corresponds to a vertical intensity edge (intensity changes horizontally).

Interpretation: Sobel produces a very large response (160) at the center — strong vertical edge.

Boundary-Based Image Segmentation

Step 1: Small 5×5 Grayscale Image (Simulated)

10	10	10	10	10
10	100	100	100	10
10	100	**255**	100	10
10	100	100	100	10
10	10	10	10	10

Step 2: Sobel Operators

	-1	0	1
$Gx =$	**-2**	0	2
	-1	0	1

	-1	**-2**	**-1**
$Gy =$	0	0	0
	1	2	1

Step 3: Apply Sobel Operator at Center (2,2)

At center (value = 255) using a 3×3 window around (2,2):

	100	100	100
$Gx =$	100	**255**	100
	100	100	100

$$Gx = (-1 \times 100 + 0 \times 100 + 1 \times 100) + (-2 \times 100 + 0 \times 255 + 2 \times 100) + (-1 \times 100 + 0 \times 100 + 1 \times 100) = 0$$

$$Gy = (-1 \times 100 + -2 \times 100 + -1 \times 100) + (0 \times 100 + 0 \times 255 + 0 \times 100) + (1 \times 100 + 2 \times 100 + 1 \times 100) = 0$$

Boundary-Based Image Segmentation

Gradient Magnitude:

$$G = \sqrt{G_x^2 + G_y^2} = \sqrt{(0^2 + 0^2)} = 0$$

No edge at the exact center — because it's surrounded by symmetrical high values.

Let's another pixel for contrast

Window around (1,2):

10	100	100
10	**100**	255
10	100	100

10	10	10	10	10
10	**100**	**100**	100	10
10	**100**	**255**	100	10
10	**100**	**100**	100	10
10	10	10	10	10

	-1	0	1
$Gx =$	**-2**	0	2
	-1	0	1

	-1	**-2**	**-1**
$Gy =$	0	0	0
	1	2	1

Gx

$$= (-1 \times 10 + 0 \times 100 + 1 \times 100) + (-2 \times 10 + 0 \times 100 + 2 \times 255) + (-1 \times 10 + 0 \times 100 + 1 \times 100) = 0$$

Gy

$$= (-1 \times 10 - 2 \times 100 - 1 \times 100) + 0 + (1 \times 10 + 2 \times 100 + 1 \times 100) = 670$$

$$G = \sqrt{G_x^2 + G_y^2} = \sqrt{(670^2)} \approx 670 \rightarrow \text{Strong edge}$$

Boundary-Based Image Segmentation

Step 4: Edge Map (Sketch Result)

	0	0	0	0	0
	0	**1**	**1**	**1**	0
$I =$	0	**1**	**0**	**1**	0
	0	**1**	**1**	**1**	0
	0	0	0	0	0

- 1 means edge detected (high gradient).
- Boundary around the central bright object is highlighted.

Applications

- Object boundary tracing
- Medical imaging (tumor outlines)
- Lane detection in self-driving cars

Face Recognition using Local Patterns of Gradients (LPG)

Background: Local Patterns of Gradients (LPG)

- LPG is similar to Local Binary Patterns but works on **gradient magnitudes or orientations** instead of raw pixel intensities.

- You compute gradients at each pixel, then encode patterns from neighboring gradients.

Problem Setup

Consider a **3×3 grayscale image patch**:

$$I = \begin{bmatrix} 50 & 80 & 80 \\ 50 & \mathbf{80} & 80 \\ 50 & 50 & 50 \end{bmatrix}$$

Face Recognition using Local Patterns of Gradients (LPG)

Step 1: Compute gradients using simple Sobel operators

Approximate horizontal (G_x) and vertical (G_y) gradients at center pixel (1,1).

$$Gx = (I_{1,2} + 2I_{1,2} + I_{2,2}) - (I_{1,0} + 2I_{1,0} + I_{2,0})$$
$$= (80 + 2 * 80 + 80) - (50 + 2 * 50 + 50) = 120$$

$$\begin{bmatrix} 50 & 80 & 80 \\ 50 & \mathbf{80} & 80 \\ 50 & 50 & 50 \end{bmatrix}$$

$$G_y = (I_{0,1} + 2I_{0,1} + I_{0,2}) - (I_{2,1} + 2I_{2,1} + I_{2,2})$$
$$= (80 + 2 * 80 + 80) - (50 + 2 * 50 + 50) = 120$$

Step 2: Compute gradient magnitude and orientation

- Gradient magnitude:

$$G = \sqrt{G_x^2 + G_y^2} = \sqrt{120^2 + 120^2} \approx 170$$

- Gradient orientation:

$$\theta = arctan(Gy/Gx) = arctan(1) = 45^o$$

Face Recognition using Local Patterns of Gradients (LPG)

Step 3: Compute gradient patterns for neighbors

For neighbors around center (positions 8 neighbors), compute gradient magnitudes and compare to center.

Neighbor	Gradient Magnitude	Compare to center (170)	Binary bit (1 if ≥ center else 0)
Top-left	100	100 < 170	0
Top	150	150 < 170	0
Top-right	180	180 ≥ 170	1
Right	200	200 ≥ 170	1
Bottom-right	160	160 < 170	0
Bottom	140	140 < 170	0
Bottom-left	130	130 < 170	0
Left	110	110 < 170	0

Thank You

Lecture 5.6

Image Segmentation

Edges Detection: Prewitt, Roberts

Instructor Name:
Professor Dr. Jahangir Alam
Professor, Dept. of CSE, BAUST

Prewitt Edge Detection of Image

$$\begin{bmatrix} 10 & 50 & 50 \\ 10 & \mathbf{50} & 50 \\ 10 & 50 & 50 \end{bmatrix}$$

Prewitt kernels:

$$G_x^p = M_{edge}(i,j) = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y^p = M_{edge}(i,j) = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

$$R_x^p = \sum M_{point}(i,j) \times I(i,j)$$
$$= (-1)(10) + 0(50) + 1(50) + (-1)(10) + 0(50) + 1(50)$$
$$+ (-1)(10) + 0(50) + 1(50) = 120$$

Positive response → **edge detected horizontally**.

$$R_y^p = \sum M_{point}(i,j) \times I(i,j)$$
$$= (-1)(10) + (-1)(50) + (-1)(50) + 0(10) + 0(50) + 0(50)$$
$$+ (1)(10) + 1(50) + 1(50) = 0$$

Positive response → **edge detected vertically**.

Prewitt Edge Detection of Image

Gradient magnitude:

$$|R| = \sqrt{(120)^2 + (0)^2} = 120$$

$$\theta = tan^{-1}2(R_y, R_x) = tan^{-1}2(0,120) = 0^o$$

Magnitude: 120 (smaller than Sobel because Prewitt doesn't weight the center row by 2).

Interpretation: Prewitt also detects the vertical edge, but with lower amplitude than Sobel.

Prewitt Edge Detection Algorithm

Very similar to Sobel but slightly simpler and computationally lighter.

To detect edges in an image (regions where intensity changes sharply) using first-order derivatives in both horizontal and vertical directions.

Step 1: Prewitt Operator Masks:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}, \qquad G_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Prewitt Edge Detection Algorithm

Step 2: Convolution

For each pixel, the mask with the 3×3 region centered at that pixel and sum the results.

$$Gx = I * Mx,$$

$$Gy = I * My$$

Step 3: Compute Gradient Magnitude

$$G = \sqrt{G_x^2 + G_y^2}$$

Or

$$G = |\, Gx \,| + |\, Gy \,|$$

This gives the **edge strength** at each pixel — large values → strong edges.

Prewitt Edge Detection Algorithm

Step 4: Compute Gradient Direction (Optional)

$$\theta = tan^{-1}\left(\frac{G_y}{G_x}\right)$$

This gives the **orientation of the edge**.

Step 5: Thresholding

Choose a threshold T to keep only significant edges:

$$E(x,y) = \begin{cases} 1 & |\, G(x,y)\,| \geq T \\ 0 & |\, G(x,y)\,| < T \end{cases}$$

This gives a binary edge map.

Prewitt Edge Detection Algorithm steps

Input Image

↓

Convert to Grayscale

↓

Gradient Calculation

(Compute edge strength and direction using Prewitt Gradient (Gx, Gy))

↓

Edge Magnitude & Direction

↓

Threshold |G| → Binary Edge Map

↓

Output: Edge Detected Image

Prewitt Edge Detection Algorithm

Advantages

- Simple and fast to compute
- Good for detecting both horizontal and vertical edges
- Computationally lighter than Sobel (no weight of 2 in center row/column)

Applications

Application	Example
Object Boundary Detection	Shape extraction
Medical Imaging	Edge-based segmentation
Computer Vision	Preprocessing for feature extraction
Robotics	Path and object edge detection

Prewitt Edge Detection Algorithm

Step 1: Apply Prewitt Operator at center pixel (2,2)

Extract the 3×3 region around pixel (2,2):

Region =

50	50	50
50	90	50
50	50	50

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \qquad G_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Convolve Gx:

Gx =(-1×50 + 0×50 + 1×50) +(-1×50 + 0×90 + 1×50)
+(-1×50 + 0×50 + 1×50) = 0

Convolve Gy:

Gy =(1×50 + 1×50 + 1×50) +(0×50 + 0×90 + 0×50)
+(-1×50 + -1×50 + -1×50)= 150 + 0 - 150 = 0

Prewitt Edge Detection Algorithm

Gradient Magnitude:

$$G = \sqrt{G_x^2 + G_y^2} = \sqrt{(0^2 + 0^2)} = 0$$

So, the center pixel is not an edge.

Step 2: Apply Prewitt at (1,2)

Region:

10	10	10
50	50	50
50	90	50

Gx=(-1×10 + 0×10 + 1×10) +(-1×50 + 0×50 + 1×50) +(-1×50 + 0×90 + 1×50)= 0 + 0 + 0 = 0

Gy=(1×10 + 1×10 + 1×10) +0 +(-1×50 + -1×90 + -1×50)= 30 - 190 = -160

Prewitt Edge Detection Algorithm

$$G = \sqrt{G_x^2 + G_y^2} = \sqrt{(0^2 + (-160)^2)} = 160$$

So, pixel (1,2) is part of an edge.

•The Prewitt operator is useful for detecting edges based on intensity gradients.
•It is simpler and faster than Sobel (which gives more weight to central pixels).
•Works well for general horizontal/vertical edge detection.

pixel	Gx	Gy	G	Edge?
(2,2)	0	0	0	No
(1,2)	0	-160	160	Yes

Roberts Cross Edge Detection Algorithm

Step 1: Roberts Cross Operator Masks

$$Gx = \begin{bmatrix} +1 & 0 \\ 0 & -1 \end{bmatrix}, Gy = \begin{bmatrix} 0 & +1 \\ -1 & 0 \end{bmatrix}$$

Step 2: Gradient Computation: For each pixel:

$$Gx = I(x, y) - I(x + 1, y + 1)$$
$$Gy = I(x + 1, y) - I(x, y + 1)$$

Step 3: Compute Gradient Magnitude

$$G = \sqrt{G_x^2 + G_y^2}$$ Or approximate with $G = |Gx| + |Gy|$

The edge strength at each pixel — large values → strong edges.

Step 4: Compute Gradient Direction (Optional): $\theta = tan^{-1}\left(\frac{G_y}{G_x}\right)$

This gives the orientation of the edge.

Roberts Cross Edge Detection Algorithm

Step 5: Thresholding

Choose a threshold T to keep only significant edges:

$$E(x,y) = \begin{cases} 1 & |\, G(x,y)\,| \geq T \\ 0 & |\, G(x,y)\,| < T \end{cases}$$

This gives a binary edge map.

Advantages

✓ Simple and fast

✓ Small 2×2 kernels → quick computation

✓ Detects fine diagonal edges effectively

Application:

- Edge detection in high-resolution images.
- Used in real-time video processing.

Roberts Cross Edge Detection Algorithm

We'll compute the edge at each **top-left corner of a 2×2 window**.

10	10	10	10
10	50	50	10
10	50	50	10
10	10	10	10

10	**10**	10	10
10	**50**	50	10
10	50	50	10
10	10	10	10

For pixel at (1,1):

$Gx = 10 - 50 = -40, Gy = 10 - 10 = 40,$

$G = \sqrt{(-40)^2 + 0^2} = 40$

10	**10**	**10**	10
10	**50**	**50**	10
10	50	50	10
10	10	10	10

***For pixel at* (1, 2):**

$Gx = 10 - 50 = -40, Gy = 50 - 10 = 40$ $G = \sqrt{(-40)^2 + 40^2} \approx 56.6$

10	10	10	10
10	**50**	**50**	10
10	**50**	**50**	10
10	10	10	10

For pixel at (2,2):

$Gx = 50 - 50 = 0, Gy = 50 - 50 = 0, G = 0$

Roberts Cross Edge Detection Algorithm

10	10	10	10
10	50	**50**	**10**
10	50	**50**	**10**
10	10	10	10

For pixel at (2,3):

$$Gx = 50 - 10 = 40, Gy = 50 - 10 = 40,$$

$$G = \sqrt{40^2 + 40^2} = 56.6$$

Edge Magnitude Map

(Values near **boundaries** show high gradients — strong edges.)

	(1,1)	(1,2)	(1,3)
(2,1)	40	56.6	56.6
(3,1)	…	…	…

Interpretation

- The **edges** appear where there's a **sudden change** in intensity (like between 10 and 50).
- Roberts operator detects **diagonal edges**, unlike Sobel or Prewitt which use horizontal/vertical masks.
- Because it uses **2×2 kernels**, it's **very sensitive to noise**, but gives **fine edge detail**.

Thank You

Lecture 5.7

Image Segmentation

Edges Detection: Canny

Instructor Name:
Professor Dr. Jahangir Alam
Professor, Dept. of CSE, BAUST

Canny Edge Detection of Image

Canny = (a) smooth → (b) gradient (Sobel) → (c) non-maximum suppression → (d) double threshold + hysteresis.

We'll do these steps numerically for the center.

a) Gaussian smoothing (3×3, σ≈1)

Kernel (normalized):

$$G = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Canny Edge Detection of Image

Compute smoothed values for the 3×3 neighborhood centered at (2,2).

$$I(i,j) = \begin{bmatrix} 10 & 10 & 10 \\ 10 & \mathbf{50} & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

Kernel

$$G = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

$$R(2,2) = \frac{1}{16}(1 \times 10 + 2 \times 10 + 1 \times 10 + 2 \times 10 + 4 \times 50 + 2 \times 10 + 1 \times 10 + 2 \times 10 + 1 \times 10) \approx 14$$

$$R(1,1) = \frac{1}{16}(4 \times 10 + 2 \times 10 + 2 \times 10 + 1 \times 50) \approx 8$$

$$R(1,3) \approx 8, R(2,1) \approx 7, R(2,3) \approx 7,$$
$$R(3,1) \approx 8, R(3,2) \approx 7, R(3,3) \approx 8$$

Smoothed image:

$$\begin{bmatrix} 8 & 7 & 8 \\ 7 & 14 & 7 \\ 8 & 7 & 8 \end{bmatrix}$$

Canny Edge Detection of Image

$$\begin{bmatrix} 10 & 50 & 50 \\ 10 & 50 & 50 \\ 10 & 50 & 50 \end{bmatrix}$$

Kernel

$$G = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

$$R_x^p = \sum M_{point}(i,j) \times I(i,j)$$
$$= (-1)(10) + 0(50) + 1(50) + (-1)(10) + 0(50) + 1(50)$$
$$+ (-1)(10) + 0(50) + 1(50) = 120$$

Positive response → **edge detected horizontally**.

$$R_y^p = \sum M_{point}(i,j) \times I(i,j)$$
$$= (-1)(10) + (-1)(50) + (-1)(50) + 0(10) + 0(50) + 0(50)$$
$$+ (1)(10) + 1(50) + 1(50) = 0$$

Positive response → **edge detected vertically**.

Canny Edge Detection of Image

Gradient magnitude:

$$|R| = \sqrt{(120)^2 + (0)^2} = 120$$

$$\theta = tan^{-1}2(R_y, R_x) = tan^{-1}2(0,120) = 0^o$$

Magnitude: |G| ≈ 120 (note smoothing reduced Sobel peak from 160 → 120).

c) Non-maximum suppression (NMS)

- Gradient direction ≈ 0° (horizontal gradient), so we compare center magnitude to neighbors along X (left/right).
- Center |G| ≈ 120 is larger than left and right neighbor magnitudes (left ≈ 0, right ≈ 0 or smaller), so it remains a local maximum → keep.

Canny Edge Detection of Image

d) Double threshold + hysteresis

Choose thresholds relative to max
(e.g., $high = 0.66 \times max \approx 79$, $low = 0.33 \times max \approx 40$).

- Center magnitude 120 > high → strong edge; neighbors between low and high become edges if connected to strong edge. Result: you get a thin, connected edge at the vertical boundary.

Interpretation: Canny yields a well localized, single-pixel-thin edge at the vertical strip — more robust and thinner compared with raw Sobel/Prewitt outputs.

Canny Edge Detection Algorithm

1. Smoothing noise,
2. Finding gradients,
3. Thinning edges,
4. Linking real edges and removing false ones.

Advantages

- ✓ Excellent detection of true edges
- ✓ Low error rate (few false positives)
- ✓ Accurate localization
- ✓ Noise-resilient (due to Gaussian smoothing)

Applications

Domain	Example
Medical Imaging	Tumor, organ, or bone boundary detection
Computer Vision	Object boundary detection
Robotics	Environment mapping
Document Analysis	Edge-based segmentation of characters or lines

Canny Edge Detection Algorithm Steps

Input Image

↓

Gaussian Filter

(Noise Reduction: Remove high-frequency noise using Gaussian filter.)

↓

Gradient Calculation

(Compute edge strength and direction using Sobel Gradient (Gx, Gy))

↓

Edge Magnitude & Direction

↓

Non-Maximum Suppression

(Thin out the edges by keeping only local maxima.)

↓

Double Threshold

(TH and TL: Classify edges into strong, weak, and non-edges.)

↓

Edge Tracking by Hysteresis

Connect weak edges that are connected to strong edges; discard isolated ones.

↓

Final Edge Map

Canny Edge Detection Algorithm

Step 1: Noise Reduction: Apply a Gaussian filter to smooth image,

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)}/2\sigma^2$$

Convolution with this filter reduces noise while preserving edge structure.

$$I_{smooth}(x,y) = \sum I(x,y) \times G(x,y)$$

Canny Edge Detection Algorithm

Step 2: Gradient Calculation

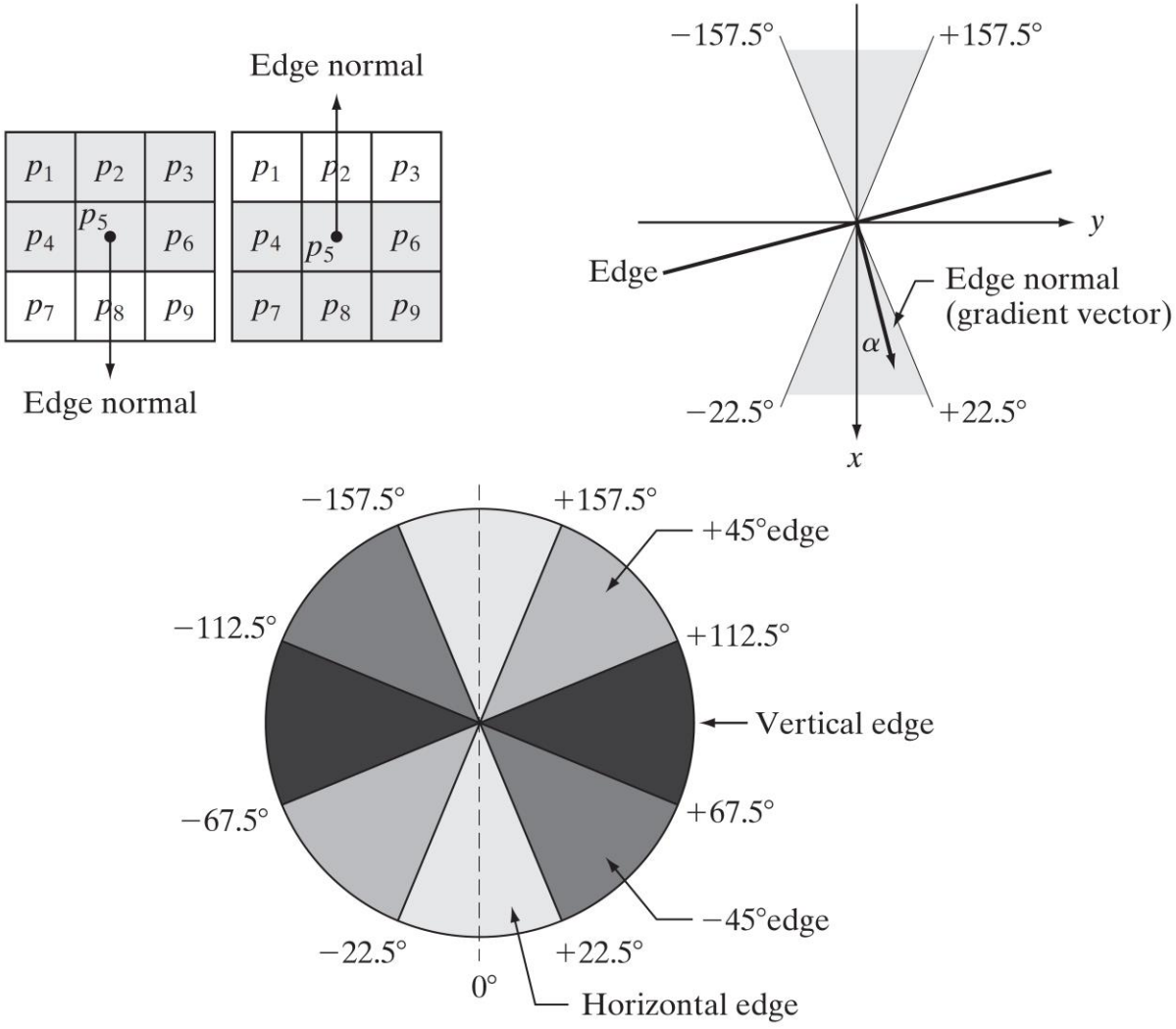
$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

$$Gx = \sum I(x,y) * Mx\,, Gy = \sum I(x,y) * My$$

$$\text{And } |G| = \sqrt{G_x^2 + G_y^2}, \theta = tan^{-1}\left(\frac{G_y}{G_x}\right)$$

Here, |G| → edge strength, and θ → edge direction (0°, 45°, 90°, 135°)

Canny edge detection Algorithm



Canny Edge Detection Algorithm

Step 3: Non-Maximum Suppression (NMS)

- For each pixel, check its gradient direction.
- If it is not the **local maximum** in that direction → set to 0.
- This thins edges to one-pixel width.

Step 4: Double Thresholding

Use two thresholds:

- High (TH) → strong edge
- Low (TL) → weak edge

$$If \ \ |G| \geq TH \Rightarrow Strong\ edge$$

$$If\ TL \leq |G| < TH \Rightarrow Weak\ edge$$

$$If \ \ |G| < TL \Rightarrow Non-edge\ (discard)$$

Canny Edge Detection Algorithm

Step 5: Edge Tracking by Hysteresis

- Keep strong edges directly.
- Keep weak edges only if connected to strong edges (continuity).
- Discard isolated weak edges (likely noise).

Comparison with Other Operators

Operator	Kernel Size	Direction Sensitivity	Noise Sensitivity	Remarks
Roberts	2×2	Diagonal	High	Simple, fast
Prewitt	3×3	Horizontal + Vertical	Moderate	Uniform weights
Sobel	3×3	Horizontal + Vertical	Low	Better smoothing
Canny	Multi-step	Multi-direction	Low	Most accurate, slower

Canny Edge Detection algorithm

Example Input Image (3×3):

$$I(i,j) = \begin{bmatrix} 10 & 10 & 10 \\ 10 & 50 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

Step 1: Gaussian smoothing (skip or approximate since image is small)

Assume no noise, so we continue with the original image for simplicity.

Step 2: Calculate gradients using Sobel operators

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Canny Edge Detection algorithm

Step 3: Compute G_x and Gy at the center pixel (position (1,1))

Use the 3×3 neighborhood (the whole image here):

$$G_x = \sum I \times G_x, \qquad Gy = \sum I \times G_y$$

Calculate:

$$G_x$$
$$= (-1) * 10 + 0 * 10 + 1 * 10 + (-2) * 10 + 0 * 50 + 2 * 10$$
$$+ (-1) * 10 + 0 * 10 + 1 * 10$$
$$= -10 + 0 + 10 - 20 + 0 + 20 - 10 + 0 + 10 = 0$$

$$G_y$$
$$= (-1) * 10 + (-2) * 10 + (-1) * 10 + 0 * 10 + 0 * 50 + 0$$
$$* 10 + 1 * 10 + 2 * 10 + 1 * 10$$
$$= -10 - 20 - 10 + 0 + 0 + 0 + 10 + 20 + 10 = 0$$

Canny Edge Detection algorithm

Step 4: Gradient magnitude and direction at center

$$G = \sqrt{G_x^2 + G_y^2} = \sqrt{0 + 0} = 0$$

Gradient direction $\theta = tan^{-1}\left(\frac{G_y}{G_x}\right) = \infty$ is undefined here (since both zero).

Interpretation:

- Center pixel is **very different from neighbors**, but Sobel gradient here is zero — due to symmetry.
- Edges would be detected on boundaries where gradient magnitude is non-zero.

Canny Edge Detection algorithm

To see edges better, slightly changed patch:

$$I(i,j) = \begin{bmatrix} 10 & 10 & 10 \\ 10 & 100 & 50 \\ 10 & 10 & 10 \end{bmatrix}$$

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Calculate Gx and Gy at center(2,2):

$$G_x = (-1) * 10 + 0 * 10 + 1 * 10 + (-2) * 10 + 0 * 100 + 2 * 50 + (-1) * 10 + 0 * 10 + 1 * 10$$
$$= -10 + 0 + 10 - 20 + 0 + 100 - 10 + 0 + 10 = 80$$

$$G_y = (-1) * 10 + (-2) * 10 + (-1) * 10 + 0 * 10 + 0 * 100 + 0 * 50 + 1 * 10 + 2 * 10 + 1 * 10$$
$$= -10 - 20 - 10 + 0 + 0 + 0 + 10 + 20 + 10 = 0$$

Canny Edge Detection algorithm

Gradient magnitude: $G = \sqrt{G_x^2 + G_y^2} = \sqrt{80^2 + 0^2} = 80$

Direction: $\theta = tan^{-1}\left(\frac{G_y}{G_x}\right) = 0^o$

Step 5: Non-maximum suppression and thresholding (conceptual)

- Check neighbors along gradient direction (horizontal here).
- Keep pixel if magnitude is local max.
- Apply thresholds to keep strong edges.

Step	Value (original)	Value (modified patch)
G_x	0	80
G_y	0	0
Gradient magnitude, G	0	80
Gradient direction, θ	undefined	0°

Canny Edge Detection algorithm

Step 1: Input Image (5×5 pixels)

This image has a bright spot (100) in the center surrounded by medium values (50), simulating an object.

10	10	10	10	10
10	50	50	50	10
10	50	**100**	50	10
10	50	50	50	10
10	10	10	10	10

Step 2: Gaussian Smoothing (Noise Reduction)

Use a small Gaussian kernel (3×3):

$$G = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

50	50	50
50	**100**	50
50	50	50

$$I_s(3,3) = G * I(3,3)$$

$$= \frac{1}{16}(1 * 50 + 2 * 50 + 1 * 50 + 2 * 50 + 4 * 100 + 2 * 50 + 1 * 50 + 2 * 50 + 1 * 50) = 62.5$$

Similarly, smooth other pixels (you can do a few manually; edges will be handled carefully).

Canny Edge Detection algorithm

10	10	10	10	10
10	50	50	50	10
10	50	**100**	50	10
10	50	50	50	10
10	10	10	10	10

50	50	50
50	**62.5**	50
50	50	50

Step 3: Compute Gradient (Using Sobel Operators)

Sobel *Gx* kernel:

$$\begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

Sobel *Gy* kernel:

$$\begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

$$Gx = (-1 \times 50) + (0 \times 50) + (1 \times 50) + (-2 \times 50) + (0 \times 62.5) + (2 \times 50) + (-1 \times 50) + (0 \times 50) + (1 \times 50) = 0$$

$$Gy = (1 \times 50) + (2 \times 50) + (1 \times 50) + (0 \times 50) + (0 \times 62.5) + (0 \times 50) + (-1 \times 50) + (-2 \times 50) + (-1 \times 50) = 0$$

Canny Edge Detection algorithm

Gradient magnitude:

$$G = \sqrt{G_x^2 + G_y^2} = \sqrt{0 + 0} = 0$$

At the edges (e.g., pixel (2,3)) gradient will be non-zero (you can compute similarly).

Step 4: Gradient Direction

$$\theta = tan^{-1}\left(\frac{Gy}{Gx}\right) = \infty$$

At (3,3) gradient is undefined → no direction.

Step 5: Non-Maximum Suppression

✓ Suppress pixels not at local maximum along gradient direction.

✓ Since center pixel gradient is zero, it gets suppressed.

Canny Edge Detection algorithm

Step 6: Double Thresholding: Choose two thresholds,

- High threshold, $T_H = 40$ and Low threshold, T_L=20

Pixels with gradient > T_H → strong edges
Pixels with gradient between T_L and T_H → weak edges
Pixels below T_L → suppressed

Step 7: Edge Tracking by Hysteresis

- Strong edges are kept.
- Weak edges connected to strong edges are kept.
- Others discarded.

Summary:

For this tiny image:

- Only pixels surrounding bright center likely get detected as edges.
- Center pixel has zero gradient → no edge.
- Outer pixels have low gradient → no edges.

Comparison Edge Detection of Image

Operator	Gx	Gy	Magnitude	Notes
Sobel	160	0	160	strong vertical edge
Prewitt	120	0	120	similar, less strong
Laplacian (4-nbr)	—	—	−35 (scalar)	second derivative; negative = bright center
Canny (after smoothing + Sobel)	120	0	120	then NMS + hysteresis → thin edge

Thank You

Lecture 5.8

Image Segmentation

Laplacian Edge Detection, LoG/ Marr-Hildreth edge detector, DoG

Instructor Name:

Prof. Dr. Jahangir Alam

Dept. of CSE, BAUST

Laplacian Edge Detection

- ✓ Second-derivative–based edge detection methods.
- ✓ To detect edges (intensity changes) by finding areas where the second derivative of intensity is high — i.e., zero-crossings in the Laplacian operator.

•**Zero-crossings** occur where a function **changes sign** (from positive to negative or vice versa).

•In **image processing**, zero-crossings are used to detect **edges** by finding points where the **second derivative of intensity crosses zero**.

Intensity	0	1	3	2	0	-1	-3	-2	0	1
Laplacian										
Zero-crossings	*				*				*	

- ✓ The Laplacian changes sign at certain points.
- ✓ These points correspond to edges in the image.

Laplacian Edge Detection

Step 1: Laplacian Operator

The **Laplacian** measures how intensity changes in all directions:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Step 2: Common discrete kernel (3×3):

$$LoG\ kernel\ (simplified) = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad or \begin{bmatrix} 0 & 1 & 0 \\ 1 & -8 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

The negative center weight ensures the filter enhances regions where intensity changes sharply.

Laplacian Edge Detection

Algorithm Steps

Step	Description
1. Grayscale Conversion	Convert color → grayscale image.
2. (Optional) Smoothing	Use Gaussian blur to reduce noise.
3. Apply Laplacian Filter	Convolve with Laplacian kernel.
4. Detect Zero-Crossings or High Values	Zero-crossings mark edges.
5. Threshold (optional)	Remove weak edges.

Applications

Field	Example
Medical Imaging	Tumor or organ boundary detection
Microscopy	Detecting cell membranes
Industrial Inspection	Detecting cracks or scratches
Computer Vision	Preprocessing for segmentation or object recognition

Laplacian Edge Detection

10	**10**	**10**	**10**	**10**
10	10	10	10	10
10	10	50	50	50
10	10	50	50	50
10	10	10	10	10

Neighborhood of center (50):

10	10	10
10	50	50
10	50	50

$$4-neighbor\ Laplacian = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

$$I = (0)(10) + (-1)(10) + (0)(10) + (-1)(10) + (4)(50) + (-1)(50) + (0)(10) + (-1)(50) + (0)(50) = 80$$

Thus, at that pixel, the Laplacian response = **+80**, indicating a **strong positive edge** (intensity jump).

Laplacian Edge Detection

10	10	10	10	10
10	10	10	10	10
10	10	50	50	50
10	10	50	50	50
10	10	10	10	10

Interpreting the Output

- Positive value (+) → center pixel brighter than neighbors → inside bright region.
- Negative value (–) → center pixel darker than neighbors → inside dark region.
- Zero value (0) → no edge.

Often, we take absolute values or use zero-crossing detection to produce a clean binary edge map.

Laplacian Edge Detection

Simplified Edge Map: Let's estimate roughly:

Pixel Position	Laplacian Output	Interpretation
Around flat area (10–10–10)	0	No edge
Near 10→50 transition	±80	Edge
Inside 50 region	0	Uniform area

Thus, the resulting **edge map** roughly looks like this:
White lines (high magnitude) mark the **edges around the 50-valued region**.

0	0	0	0	0
0	80	80	80	0
0	80	0	80	0
0	80	80	80	0
0	0	0	0	0

Laplacian of Gaussian (LoG) Edge Detection

- Apply Gaussian smoothing then Laplacian operator (second derivative).
- Detects zero-crossings in the filtered image (edges).

LoG = Smooth (Gaussian) + Differentiate (Laplacian)

The Laplacian operator alone is very sensitive to noise, so we first smooth the image with a Gaussian filter, and

Then apply the Laplacian. This combination is called the Laplacian of Gaussian (LoG).

Laplacian of Gaussian (LoG) Edge Detection

The **Marr–Hildreth Edge Detector** (proposed by David Marr & Ellen Hildreth, 1980) detects edges by finding **zero-crossings** in the **Laplacian of Gaussian (LoG)** filtered image.

It combines:

1.Noise reduction (Gaussian smoothing)

2.Edge enhancement (Laplacian operator)

3.Edge localization (Zero-crossing detection)

Laplacian of Gaussian (LoG)

$$LoG(I) = \nabla^2[G(x, y) * I(x, y)] = [\nabla^2 G(x, y)] * I(x, y)$$

where

- ∇^2 = Laplacian operator (second derivative)
- $G(x, y)$ = Gaussian smoothing function

Algorithm Steps

Step	Description
1. Convert to Grayscale	Convert image to single intensity channel.
2. Gaussian Smoothing	Reduce noise using Gaussian filter.
3. Apply Laplacian	Find second derivative of intensity.
4. Detect Zero Crossings	Pixels where Laplacian changes sign = edges.

The Marr-Hildreth edge detector

Marr and Hildreth argued

(1) that intensity changes are not independent of image scale and so their detection requires the use of operators of different sizes; and

(2) that a sudden intensity change will give rise to a peak or trough in the first derivative or, equivalently, to a zero crossing in the second derivative

The Marr-Hildreth edge detector

These ideas suggest that an operator used for edge detection should have two salient features.

First and foremost, it should be a differential operator capable of computing a digital approximation of the first or second derivative at every point in the image.

Second, it should be capable of being "tuned" to act at any desired scale, so that large operators can be used to detect blurry edges and small operators to detect sharply focused fine detail.

The Marr-Hildreth edge detector

The Marr and Hildreth argued that the most satisfactory operator fulfilling these conditions is the filter $\nabla^2 G(x,y)$, where ∇^2 is the Laplacian operator,

$(\partial^2/\partial x^2 + \partial^2/\partial y^2)$, and G is the 2-D Gaussian function

$$G(x, y) = e^{-\frac{x^2+y^2}{2\sigma^2}}$$

with standard deviation σ (sometimes σ is called the space constant).To find an expression for $\nabla^2 G(x,y)$ we perform the following differentiations:

$$\nabla^2 G(x,y) = \frac{\partial^2 G(x,y)}{\partial x^2} + \frac{\partial^2 G(x,y)}{\partial y^2} = \frac{\partial}{\partial x}\left[\frac{-x}{2\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}\right] + \frac{\partial}{\partial y}\left[\frac{-y}{2\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}\right]$$

$$\nabla^2 G(x,y) = \left[\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2}\right] e^{-\frac{x^2+y^2}{2\sigma^2}} + \left[\frac{y^2}{\sigma^4} - \frac{1}{\sigma^2}\right] e^{-\frac{x^2+y^2}{2\sigma^2}}$$

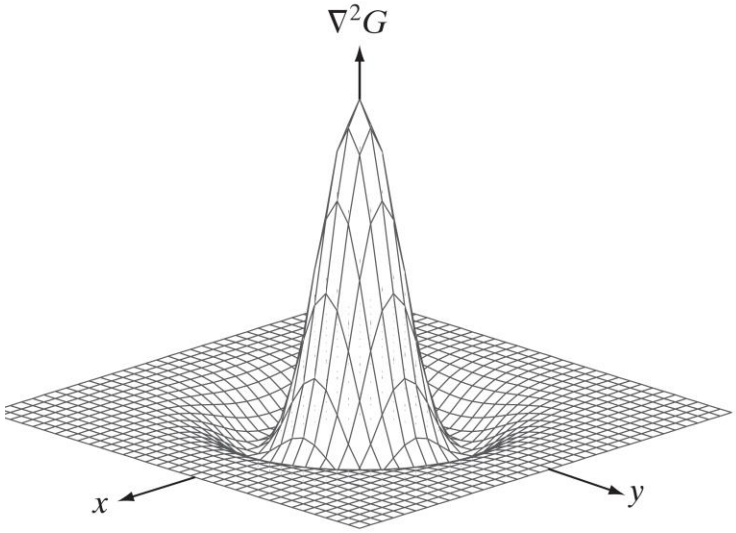
The Marr-Hildreth edge detector

Collecting terms gives the final expression:

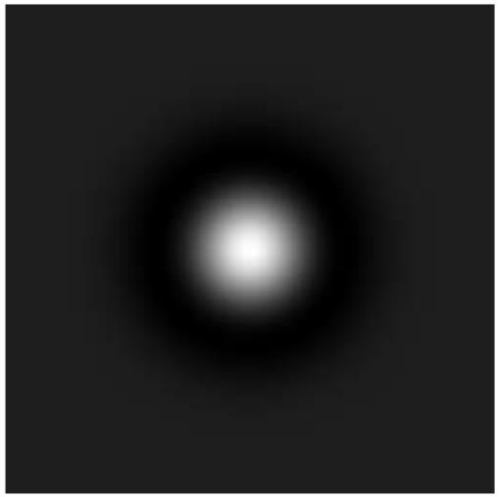
$$\nabla^2 G(x,y) = \left[\frac{x^2 + y^2 - 2\sigma^2}{\sigma^4}\right] e^{-\frac{x^2+y^2}{2\sigma^2}}$$

This expression is called the *Laplacian of a Gaussian (LoG)*.

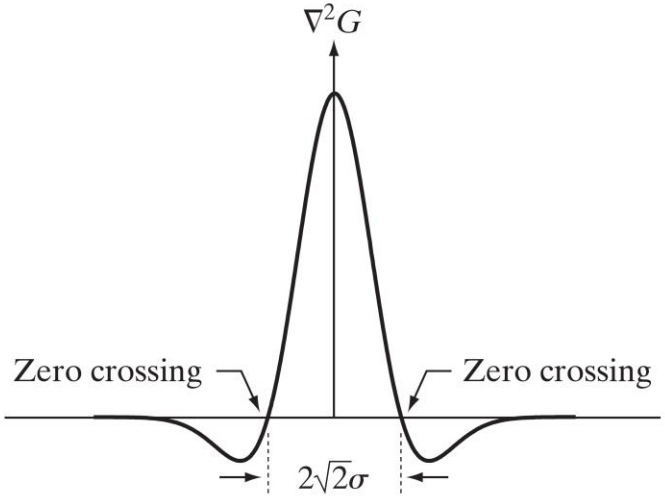
The Marr-Hildreth edge detector



(a) 3D plot of the negative of the LoG.



(b) Negative of the LoG displayed as an image.

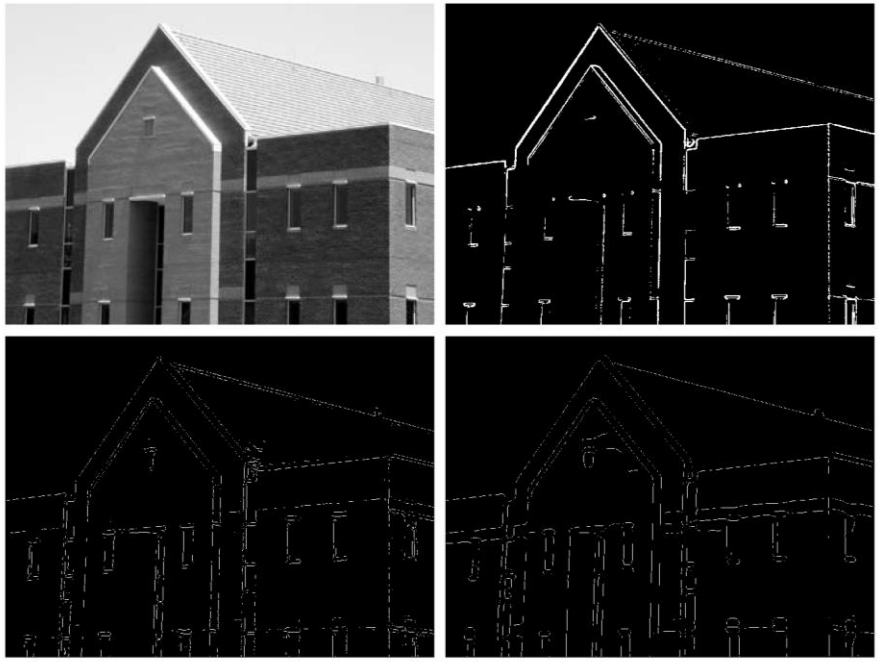


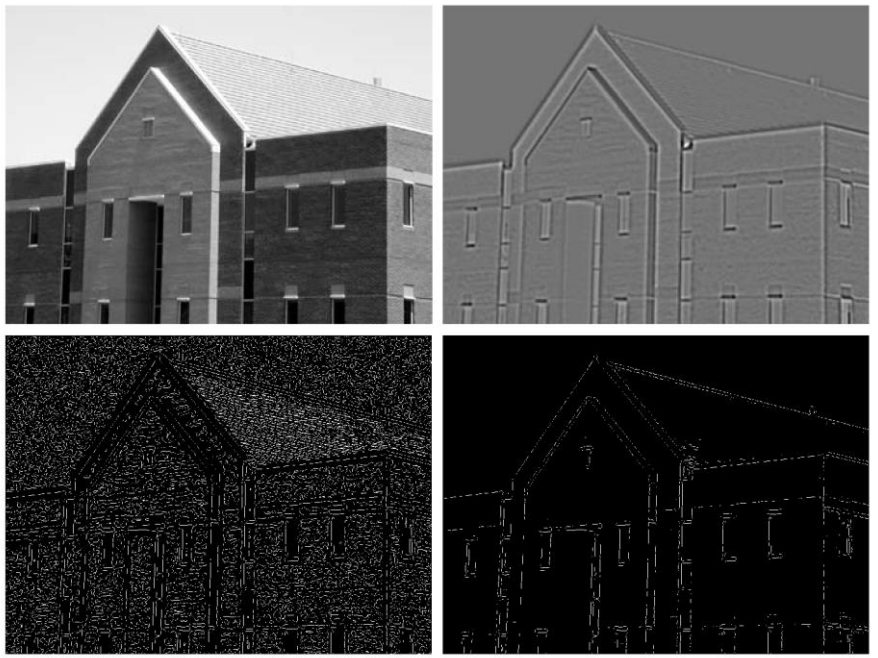
(c) Cross section of (a) showing zero crossings.

0	0	−1	0	0
0	−1	−2	−1	0
−1	−2	16	−2	−1
0	−1	−2	−1	0
0	0	−1	0	0

(d) 5 × 5 mask approximation to the shape in (a).

The Marr-Hildreth edge detector





Laplacian of Gaussian (LoG)

Step 1: Gaussian Smoothing: The **Gaussian function**,

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)}/2\sigma^2$$

It blurs the image and reduces noise.

Step 2: Laplacian Operator

The **Laplacian** measures how intensity changes in all directions:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Common discrete kernel (3×3):

$$LoG\ kernel\ (simplified) = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad or, \begin{bmatrix} 0 & 1 & 0 \\ 1 & -8 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Laplacian of Gaussian (LoG)

Step 3: Combine into LoG Operator

The **Laplacian of Gaussian kernel** (for σ = 1.0) looks like:

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$$

$$LoG(x,y) = -\frac{1}{\pi\sigma^4}\left[1 - \frac{x^2+y^2}{2\sigma^2}\right] e^{-(x^2+y^2)}/2\sigma^2$$

It has a **Mexican-hat** shape — positive center, negative surrounding.



Laplacian of Gaussian (LoG)

Advantages

- Reduces noise before detecting edges
- Detects **fine and smooth edges**
- Works in all directions (isotropic)
- Produces thin edge maps

Limitations

- More computationally expensive
- May miss weak edges (depends on σ)
- Zero-crossing detection can be unstable in noisy images

Applications

Application	Example
Medical Imaging	Edge-based organ segmentation
Object Boundary Detection	Precise shape extraction
Microscopic Image Analysis	Detecting cells and structures
Computer Vision Preprocessing	Before feature extraction

Laplacian of Gaussian (LoG)

10	10	10	10	10
10	10	10	10	10
10	10	**50**	50	50
10	10	50	50	50
10	10	10	10	10

Step 1: Gaussian Smoothing

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)}/2\sigma^2$$

A 2D **Gaussian filter** is:

$$G(x,y) = e^{-(x^2+y^2)}/2\sigma^2$$

Typical discrete 5×5 Gaussian (σ = 1) kernel:

$$\frac{1}{273}\begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}$$

We'll use the **4-neighbor Laplacian mask**:

$$4-neighbor\ Laplacian = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Laplacian of Gaussian (LoG)

10	10	10	10	10
10	10	10	10	10
10	10	**50**	50	50
10	10	50	50	50
10	10	10	10	10

We'll apply a simplified 3×3 Gaussian kernel:

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Let's smooth the center pixel (row 3, col 3 = 50) Neighborhood:

10	10	10
10	50	50
10	50	50

$$I_{G(smooth)} = \frac{1(10) + 2(10) + 1(10) + 2(10) + 4(50) + 2(50) + 1(10) + 2(50) + 1(50)}{16} = 32.5$$

After Gaussian smoothing, central pixel value ≈ **32.5**

Laplacian of Gaussian (LoG)

10	10	10	10	10
10	10	10	10	10
10	10	**50**	50	50
10	10	50	50	50
10	10	10	10	10

10	10	10
10	50	50
10	50	50

Step 2: Apply Laplacian

We'll use the **4-neighbor Laplacian mask**:

$$4-neighbor\ Laplacian = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

15	20	25
20	32.5	40
25	40	45

Let's compute Laplacian 3×3 Neighborhood after smoothing (approximate values):

$$I_{G(Laplacian)} = (0)(15) + (-1)(20) + (0)(25) + (-1)(20) + (4)(32.5) + (-1)(40) + (0)(25) + (-1)(40) + (0)(45) = 10$$

Laplacian result at (3,3) = **+10**

Laplacian of Gaussian (LoG)

Step 3: Zero-Crossing Detection

If we compute nearby pixels, some will have positive and some negative Laplacian responses. Where the result changes sign (from + to − or vice versa), an edge exists.

For our image, edges occur near the boundary between 10 and 50 regions.

Final LoG Edge Map (Conceptually)

The **edges appear around the bright 50-region** after smoothing.

Pixel	Approx Laplacian (after Gaussian)	Edge?
10→10	0	No edge
10→50	±10 to ±20	Edge
50→50	0	No edge

Difference of Gaussian (DoG) Edge Detection

DoG Edge Detection is simplified and efficient approximation of LoG method.

To detect edges by finding regions where intensity changes sharply, using the difference between two Gaussian-blurred images with different standard deviations (σ).

DoG ≈ LoG — but computationally simpler.

- ✓ Smooth image with a **Gaussian filter** at scale **σ_1 (less blur)**.
- ✓ Smooth same image with another **Gaussian filter** at a **larger scale σ_2**.

The result emphasizes **edges** and **fine details** that disappear at coarser (rough) scales.

Difference of Gaussian (DoG) Edge Detection

If $G(x, y, \sigma)$ is a Gaussian function

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)}/2\sigma^2$$

✓ Then, Subtract the two blurred images

$$DoG = G(x,y,\sigma 1) * I(x,y) - G(x,y,\sigma 2) * I(x,y)$$

$$LoG(x,y) \approx \frac{G(x,y,\sigma 1) - G(x,y,\sigma 2)}{\sigma 2 - \sigma 1}$$

Advantages

- Simpler and faster than LoG
- Reduces high-frequency noise through smoothing
- Suitable for multi-scale feature detection
- Used in SIFT (Scale-Invariant Feature Transform) for keypoint detection

The Marr-Hildreth edge detector

Marr and Hildreth [1980] noted that it is possible to approximate the LoG filter by a difference of Gaussians (DoG):

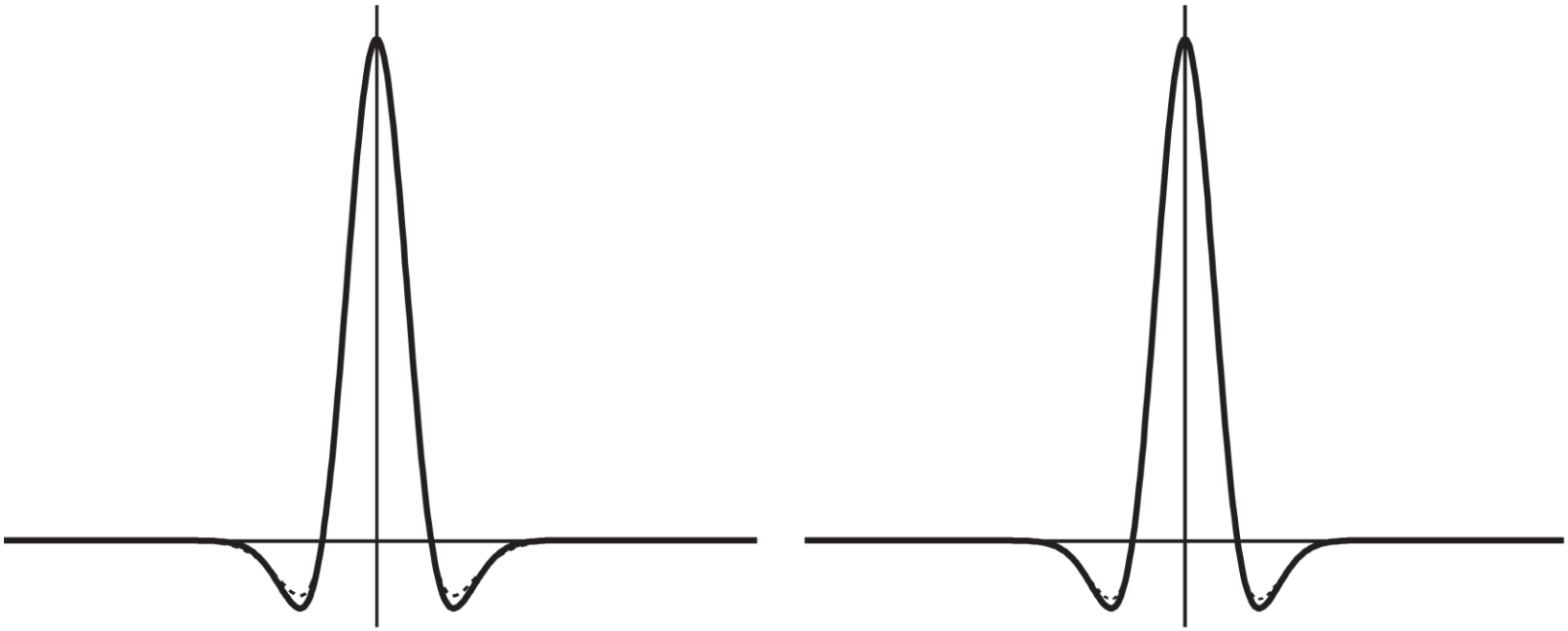
$$G(x,y) = \frac{1}{2\pi\sigma_1^2} e^{-\frac{x^2+y^2}{2\sigma_1^2}} - \frac{1}{2\pi\sigma_2^2} e^{-\frac{x^2+y^2}{2\sigma_2^2}}$$

with $\sigma_1 > \sigma 2$. To make meaningful comparisons between the LoG and DoG, the value of σ for the LoG must be selected as in the following equation so that the LoG and DoG have the same zero crossings:

$$\sigma^2 = \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2-\sigma_2^2} ln\left[\frac{\sigma_1^2}{\sigma_2^2}\right]$$

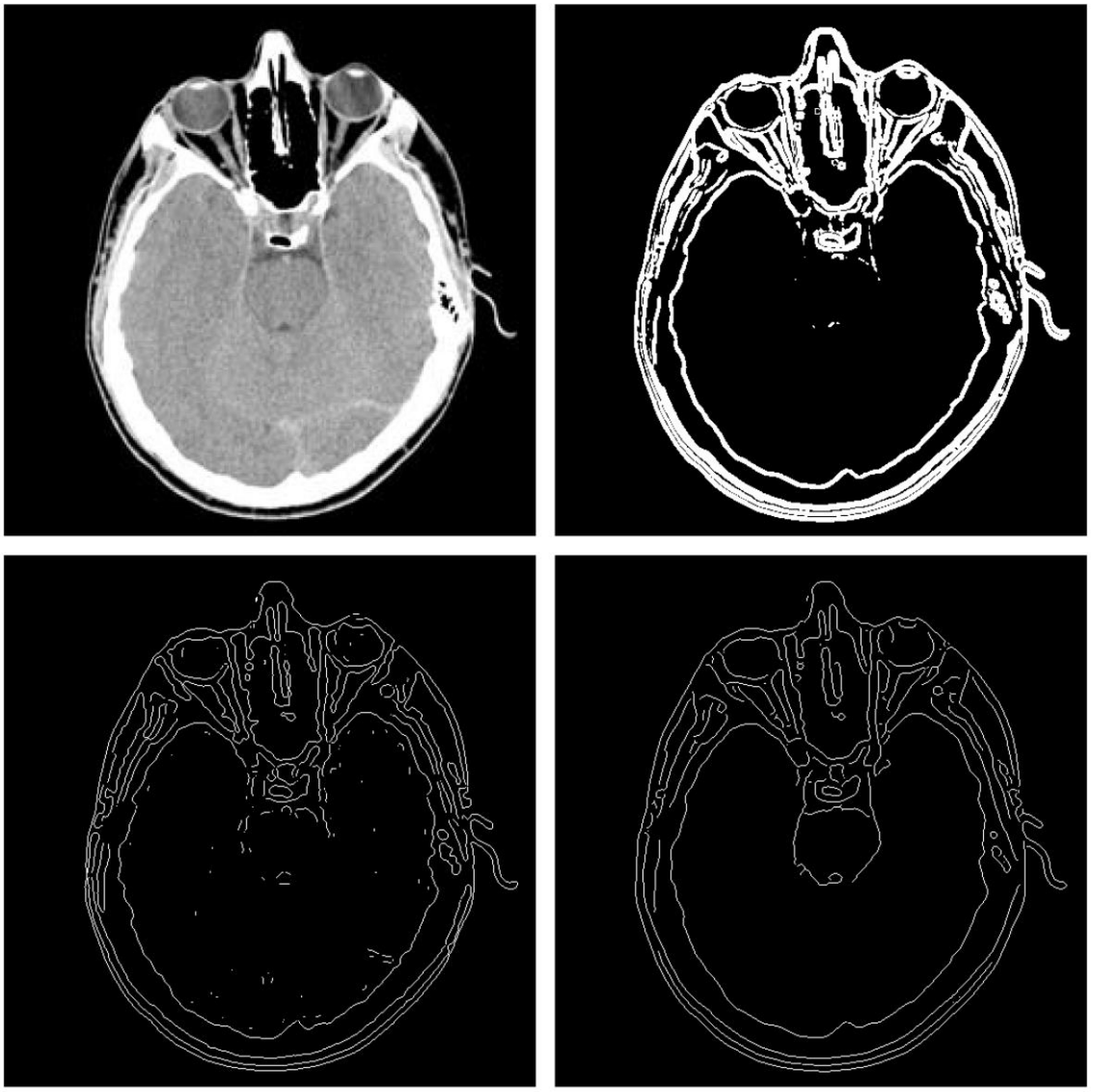
Although the zero crossings of the LoG and DoG will be the same when this value of σ is used, their amplitude scales will be different. We can make them compatible by scaling both functions so that they have the same value at the origin.

The Marr-Hildreth edge detector



(a) Negatives of the LoG (solid) and DoG (dotted) profiles using a standard deviation ratio of 1.75:1. (b) Profiles obtained using a ratio of 1.6:1.

The Marr-Hildreth edge detector



Difference of Gaussian (DoG) Edge Detection

Algorithm Steps

Step	Description
1. Input Image	Load the grayscale image.
2. Gaussian Blur #1	Apply Gaussian filter with σ_1 (e.g., 1).
3. Gaussian Blur #2	Apply Gaussian filter with σ_2 (e.g., 2).
4. Subtract	Compute DoG = $Blur_1 - Blur_2$.
5. Threshold (optional)	Highlight significant edges.
6. Output Edge Map	Display or use for further processing.

Limitations

- ✓ Edge localization slightly less precise than LoG
- ✓ Still sensitive to high noise if σ values are too small
- ✓ Choice of σ values affects results significantly

Difference of Gaussian (DoG) Edge Detection

Algorithm	Kernel Size	Noise Sensitivity	Derivative	Computation	Detects	Edge Thickness	Use Case
Sobel	3×3	Moderate	1st	Fast	Gradient edges	Thick	General edge detection
Prewitt	3×3	Moderate	1st	Fast	Gradient edges	Thick	Simpler, faster processing
Roberts	2×2	High	1st	Fast	Diagonal edges	Thin	High-res, real-time
Canny	Variable	Low (good noise suppression)	1st	Fast	Gradient edges	Thin	Best overall performance
Laplacian	Variable	High	2nd	Fast	Gradient edges	Thin	Best overall performance
LoG	Variable	Low	2nd	Moderate	Zero-crossing	Thin	Smooth & precise, Edge + blob detection
LoG	Variable	Low	2nd	Faster	Zero-crossing	Thin	Smooth & precise, Edge + blob detection

Difference of Gaussian (DoG) Edge Detection

10	10	10	10	10
10	10	10	10	10
10	10	50	50	50
10	10	50	50	50
10	10	10	10	10

We'll perform:

1. Gaussian blur with $\sigma_1 = \mathbf{1}$ (small blur)
2. Gaussian blur with $\sigma_2 = \mathbf{2}$ (large blur)
3. Subtract: **DoG = Blur(σ_1) – Blur(σ_2)**

Step 1: Gaussian Blur ($\sigma_1 = 1$)

Use a small Gaussian kernel (3×3):

$$G_{\sigma_1} = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Compute at center pixel (row 3, col 3 = 50) Neighborhood:

10	10	10
10	50	50
10	50	50

$$I_{\sigma_1} = \frac{1(10) + 2(10) + 1(10) + 2(10) + 4(50) + 2(50) + 1(10) + 2(50) + 1(50)}{16}$$

$$= 32.5$$

Smoothed intensity (σ_1) = 32.5

Difference of Gaussian (DoG) Edge Detection

10	10	10	10	10
10	10	10	10	10
10	10	50	50	50
10	10	50	50	50
10	10	10	10	10

We'll perform:

1. Gaussian blur with $\sigma_1 = \mathbf{1}$ (small blur)
2. Gaussian blur with $\sigma_2 = \mathbf{2}$ (large blur)
3. Subtract: $\mathbf{DoG = Blur(\sigma_1) - Blur(\sigma_2)}$

Step 2: Gaussian Blur ($\sigma_2 = 2$)

A larger blur smooths more (we can approximate values more evenly averaged).

Simplify with a coarser average — use equal weights (approximation):

$$G_{\sigma_2} = \frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix}$$

Compute again for same neighborhood:

$$I_{\sigma_2} = \frac{10 + 10 + 10 + 10 + 50 + 50 + 10 + 50 + 50}{9} \approx 32.2$$

Smoothed intensity (σ_2) = 32.2

Difference of Gaussian (DoG) Edge Detection

10	10	10	10	10
10	10	10	10	10
10	10	50	50	50
10	10	50	50	50
10	10	10	10	10

We'll perform:

1. Gaussian blur with $\sigma_1 = \mathbf{1}$ (small blur)
2. Gaussian blur with $\sigma_2 = \mathbf{2}$ (large blur)
3. Subtract: $\mathbf{DoG = Blur(\sigma_1) - Blur(\sigma_2)}$

Step 3: Difference of Gaussian

$$DoG = I_{\sigma_1} - I_{\sigma_2} = 32.5 - 32.2 = 0.3$$

Small value → almost flat region.

10	10	10	10	10
10	10	10	10	10
10	10	50	50	50
10	10	50	50	50
10	10	10	10	10

Now let's check an edge pixel (around where 10 meets 50).
At edge pixel (row 3, col 2 = 10)
Neighborhood:

$$G_{\sigma_1} = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

For $\sigma_1 = 1$:

$$I_{\sigma_1} = (10 + 20 + 10 + 20 + 40 + 100 + 10 + 20 + 50)/16 = 17.5$$

Difference of Gaussian (DoG) Edge Detection

$$G_{\sigma_2} = \frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix}$$

10	10	10	10	10
10	10	10	10	10
10	10	50	50	50
10	10	50	50	50
10	10	10	10	10

For $\sigma_2 = 2$:

$$I_{\sigma_2} = \frac{10+10+10+10+10+50+10+10+50}{9} \approx 18.9$$

$$DoG = 17.5 - 18.9 = -1.4$$

Strong negative response → edge detected.

Pixel Zone	DoG Value	Edge?
Flat (10→10)	≈ 0	No edge
Edge (10→50)	±1–2	Edge
Flat (50→50)	≈ 0	No edge

Key Notes

- ✓ DoG ≈ LoG (computationally cheaper)
- ✓ Often used in SIFT to detect keypoints.
- ✓ Works best when $\sigma_2 \approx 1.6\sigma_1$

Combining the gradient with thresholding



Thank You

Lecture 5.7

Image Segmentation

Region-Based Segmentation, Region Splitting and Merging

Instructor Name:

Prof. Dr. Jahangir Alam

Dept. of CSE, BAUST

Region-Based Segmentation

Region-based segmentation is a technique in Image Processing where an image is divided into regions based on **similarity criteria** such as intensity, color, or texture.

1. **Region Growing**
   - ✓ Start from seed points and grow region if neighbor pixels are similar.
2. **Region Splitting and Merging**
   - ✓ Split large heterogeneous regions; merge small and similar homogeneous adjacent regions.

Example:

- ✓ Used in MRI brain segmentation, ultrasound organ detection, etc.
- ✓ Segmenting brain tissues (gray matter, white matter) in MRI.

Key Concepts Region-Based Segmentation

1. **Seed Points**
   - ✓ The starting points for region growth.
   - ✓ Can be manually selected or automatically chosen (e.g., intensity peaks).
2. **Similarity Criterion**
   - ✓ Defines whether a pixel should be added to a region.
   - ✓ Example (grayscale intensity):

$$| I(x, y) - \mu_R | < T$$

     - $I(x, y)$ = intensity of candidate pixel
     - μ_R = mean intensity of the current region
3. **Connectivity**
   - ✓ Determines which neighbors are considered:
     - 4-connectivity → up, down, left, right
     - 8-connectivity → diagonals included
4. **Stopping Condition**
   - ✓ The algorithm stops when no new neighboring pixels satisfy the similarity criterion.

Region-Based Segmentation

Region growing

Algorithm Steps

1. **Select seed point(s)/**starting pixel (or set of pixels)
2. **Initialize region** with seed(s)
3. **Check neighbors** (4-neighborhood or 8-neighborhood)
4. **Apply similarity condition**
5. **Add qualifying pixels** to region
6. **Update region statistics (mean, variance)**
7. Repeat until no more pixels can be added

Region-Based Segmentation

Advantages

- Produces connected regions, suitable for object extraction.
- Handles irregular shapes.
- Can incorporate multiple features (intensity, color, texture).

Limitations

- Sensitive to seed selection.
- Can leak into neighboring regions if edges are weak.
- Sensitive to noise; smoothing may be required.
- Computationally more intensive for large images.

Applications

- Medical Imaging: Tumor or organ segmentation.
- Remote Sensing: Land and water separation.
- Industrial Vision: Defect detection.
- Object Extraction in general computer vision tasks.

Region-Based Segmentation

Step 1: Grayscale Image (5×5)

	50	51	52	180	182
	49	50	51	181	179
$I =$	50	52	49	180	180
	180	180	179	50	49
	181	182	180	48	50

Target: Segment the low-intensity region (dark pixels) from high-intensity background.

Step 2: Choose Seed Pixel

We select pixel at position (0,0) with intensity = 50 as the seed for region growing.

- We'll define a similarity threshold of ±3.
- This means any pixel from 47–53 can join the region.

Region-Based Segmentation

Step 3: Region Growing Procedure

We'll simulate Breadth-First Search (BFS) to grow the region:

Initialization:

- Seed Region = { (0,0) }
- Intensity Range = 47 to 53

Iteration:

Check 4-neighbors (up, down, left, right) of all region pixels. Add neighbors if intensity ∈ [47, 53].

Step 1:

From (0,0) = 50 → Check neighbors:

- (0,1) = 51 → Add
- (1,0) = 49 → Add

Region: {(0,0), (0,1), (1,0)}

	50	51	52	180	182
	49	50	51	181	179
I =	50	52	49	180	180
	180	180	179	50	49
	181	182	180	48	50

Region-Based Segmentation

Step 2:
From (0,1) = 51:
- (0,2) = 52 → Add
- (1,1) = 50 → Add

Region: {(0,0), (0,1), (1,0), (0,2), (1,1)}

50	51	52	180	182
49	50	51	181	179
50	52	49	180	180
180	180	179	50	49
181	182	180	48	50

Step 3:
From (1,0) = 49:
- (2,0) = 50 → Add

Region: + (2,0)

Step 4:
From (2,0) = 50:
- (2,1) = 52 → Add

Region: + (2,1)

Step 5:
From (2,1) = 52:
- (2,2) = 49 → Add

...
We continue until no more neighbors satisfy the range.

Region-Based Segmentation

Final Segmented Region, Segmented pixels (marked 1):

Region grew to include the bottom-left corner as well: pixels (3,3), (3,4), (4,3), (4,4) are also between 48–50.

	1	1	1	0	0
	1	1	0	0	0
$I =$	1	1	1	0	0
	0	0	0	1	1
	0	0	0	1	1

Applications

- Medical images (e.g., tumor areas)
- Satellite imagery
- Fingerprint & biometric segmentation

Region-Based Segmentation

Given: A small 5×5 grayscale image:

52	53	54	150	152
50	51	55	149	151
49	50	53	148	150
48	49	52	200	205
47	48	50	198	202

Step 1: Select seed point

- Choose seed pixel at position (2,2) with intensity 51.

Step 2: Define similarity criterion

- Include neighboring pixels with intensity difference ≤ 5 compared to seed or grown region mean.

Step 3: Initialize region

- Region = {(2,2)} with intensity mean = 51.

Region-Based Segmentation

52	53	54	150	152
50	51	55	149	151
49	50	53	148	150
48	49	52	200	205
47	48	50	198	202

Step 4: Check neighbors of (2,2)

Neighbors (4-connected): (1,2), (2,1), (2,3), (3,2)

- (1,2) = 53, difference $|53 - 51| = 2 \leq 5 \rightarrow$ Add to region
- (2,1) = 50, difference $|50 - 51| = 1 \leq 5 \rightarrow$ Add to region
- (2,3) = 55, difference $|55 - 51| = 4 \leq 5 \rightarrow$ Add to region
- (3,2) = 50, difference $|50 - 51| = 1 \leq 5 \rightarrow$ Add to region

Region = {(2,2), (1,2), (2,1), (2,3), (3,2)}

Step 5: Update region mean

$$\text{Mean}=(51+53+50+55+50)/5=51.8$$

Region-Based Segmentation

Step 6: Check neighbors of newly added pixels

Check neighbors of (1,2):

- (1,1) = 52, |52 - 51.8| = 0.2 ≤ 5 → Add
- (1,3) = 54, |54 - 51.8| = 2.2 ≤ 5 → Add

Check neighbors of (2,1):

- (3,1) = 49, |49 - 51.8| = 2.8 ≤ 5 → Add

Check neighbors of (2,3):

- (3,3) = 53, |53 - 51.8| = 1.2 ≤ 5 → Add

Check neighbors of (3,2):

- (4,2) = 49, |49 - 51.8| = 2.8 ≤ 5 → Add

52	53	54	150	152
50	51	55	149	151
49	50	53	148	150
48	49	52	200	205
47	48	50	198	202

Add these to region:

{(1,1),(1,3),(3,1),(3,3),(4,2)}

Region now has 10 pixels.

Step 7: Update mean

Mean=2(52+54+49+53+49)/10=514/10=51.6

Region-Based Segmentation

Step 8: Repeat checking neighbors

For example, neighbors of (1,1):

- (1,0) (out of bounds), (0,1) (out of bounds), (2,1) (already in region), (1,2) (already in region)

Neighbors of (1,3):

- (1,4) = 150, $|150 - 51.6| = 98.4 > 5$ (stop)
- etc.

You continue checking neighbors only if difference ≤ 5.

Step 9: Stop when no new pixels satisfy the criterion.

Final Result:

The region includes all connected pixels around seed point with similar intensities (close to ~52), excluding high-intensity pixels (150+), effectively segmenting the low-intensity region.

Region Growing: Grouping Pixels by Similarity

A small 4×4 grayscale image with pixel intensities:

	1	2	3	4
1	45	47	150	152
2	46	48	148	149
3	44	45	151	150
4	43	44	149	148

Step 1: Select seed point

- Seed pixel at position (1,1) with intensity = 45.

Step 2: Define similarity criterion

- Include neighbor pixels with intensity difference ≤ 5 from the current region mean.

Step 3: Initialize region

- Region = {(1,1)}
- Mean intensity = 45

Region Growing: Grouping Pixels by Similarity

A small 4×4 grayscale image with pixel intensities:

	1	2	3	4
1	45	47	150	152
2	46	48	148	149
3	44	45	151	150
4	43	44	149	148

Step 4: Check 4-connected neighbors of (1,1):

- (1,2) = 47 → |47 - 45| = 2 ≤ 5 → Add
- (2,1) = 46 → |46 - 45| = 1 ≤ 5 → Add

Region = {(1,1), (1,2), (2,1)}

Step 5: Update mean intensity:

$$Mean = (45 + 47 + 46)/3 = 46$$

Region Growing: Grouping Pixels by Similarity

A small 4×4 grayscale image with pixel intensities:

	1	2	3	4
1	45	47	150	152
2	46	48	148	149
3	44	45	151	150
4	43	44	149	148

Step 6: Check neighbors of newly added pixels

- Neighbors of (1,2): (1,3) = 150 → |150 - 46| = 104 > 5 → **Exclude**
- Neighbors of (2,1): (3,1) = 44 → |44 - 46| = 2 ≤ 5 → Add
- Neighbors of (2,1): (2,2) = 48 → |48 - 46| = 2 ≤ 5 → Add

Region now:

$$\{(1,1),(1,2),(2,1),(3,1),(2,2)\}$$

Region Growing: Grouping Pixels by Similarity

Step 7: Update mean intensity:

$$\text{Mean}=45+47+46+44+485=2305=46$$

Step 8: Continue checking neighbors

- Neighbors of (2,2): (3,2) = 45 → |45 - 46| = 1 ≤ 5 → Add
- Neighbors of (3,1): (4,1) = 43 → |43 - 46| = 3 ≤ 5 → Add

Region now:

$$\{(1,1),(1,2),(2,1),(3,1),(2,2),(3,2),(4,1)\}$$

Step 9: Update mean intensity:

$$Mean = \frac{230 + 45 + 43}{7} \approx 45.43$$

Region Growing: Grouping Pixels by Similarity

Step 10: Check neighbors of newly added pixels

- Neighbors of (3,2): (4,2) = 44 → |44 - 45.43| = 1.43 ≤ 5 → Add
- Neighbors of (4,1): (4,2) already added.

Region now:

{(1,1),(1,2),(2,1),(3,1),(2,2),(3,2),(4,1),(4,2)}

Step 11: Update mean intensity:

$$Mean = \frac{318 + 44}{8} = 45.25$$

Region Growing: Grouping Pixels by Similarity

Step 12: Check remang neighbors (3,3), (4,3), etc.

- (3,3) = 151 → |151 - 45.25| ≫ 5 → **Exclude**
- (4,3) = 149 → |149 - 45.25| ≫ 5 → **Exclude**

Step 13: Stop — no more pixels satisfy criterion

Final Region:

Pixels with intensities around 45-48 are grouped together, separating them from higher intensity pixels (around 150).

Region-Based and Feature-Based Segmentation

Given Image (6×6 pixels)

52	54	55	200	201	202
53	54	56	198	200	203
52	55	57	199	202	204
10	11	12	150	152	153
12	11	13	148	150	155
10	12	14	151	149	154

1. Region-Based Segmentation (Region Growing)

Target: Segment the image into regions where pixel intensities are similar and connected.

Step:

- Pick a seed pixel and grow the region by adding neighboring pixels with intensity difference ≤ threshold.

Example:

- Seed pixel: (0,0) with intensity 52 and Threshold: 5

Region-Based and Feature-Based Segmentation

Growing region around (0,0):

- Check neighbors of 52:
  - (0,1): 54 → $|54-52|=2 \leq 5$ → include
  - (1,0): 53 → $|53-52|=1 \leq 5$ → include
  - (1,1): 54 → $|54-52|=2 \leq 5$ → include
  - (0,2): 55 → $|55-52|=3 \leq 5$ → include

- For newly added pixels, repeat checking neighbors similarly.
- The first 3×3 block of pixels in the top-left corner (values around 52-57) gets grouped as **Region 1**.

Region-Based and Feature-Based Segmentation

Similarly, start from (0,3) intensity 200:

- Grow region with threshold 5:
  - Pixels with intensities ~198 to 204 grouped as **Region 2**.

Similarly, bottom-left pixels (~10-14) grouped as **Region 3**.

Similarly, bottom-right pixels (~148-155) grouped as **Region 4**.

Result:

R1	R1	R1	R2	R2	R2
R1	R1	R1	R2	R2	R2
R1	R1	R1	R2	R2	R2
R3	R3	R3	R4	R4	R4
R3	R3	R3	R4	R4	R4
R3	R3	R3	R4	R4	R4

Feature-Based Segmentation (Clustering Using Pixel Intensity)

Target: Segment based on features (pixel intensities) regardless of spatial connectivity.

Method: Use k-means clustering with k=4 (4 clusters).

Step 1: Flatten pixels intensities:

52	54	55	200	201	202
53	54	56	198	200	203
52	55	57	199	202	204
10	11	12	150	152	153
12	11	13	148	150	155
10	12	14	151	149	154

Initial cluster centers (randomly selected):

- C1 = 12
- C2 = 55
- C3 = 150
- C4 = 200

Feature-Based Segmentation (Clustering Using Pixel Intensity)

Assign each pixel to nearest center by intensity difference:

- Pixels near 10-14 → Cluster C1
- Pixels near 52-57 → Cluster C2
- Pixels near 148-155 → Cluster C3
- Pixels near 198-204 → Cluster C4

Result clusters (segmented regions):

Note:

•Unlike region growing, feature-based segmentation does NOT require spatial connectivity. It groups pixels based on similar intensity regardless of position.

C2	C2	C2	C4	C4	C4
C2	C2	C2	C4	C4	C4
C2	C2	C2	C4	C4	C4
C1	C1	C1	C3	C3	C3
C1	C1	C1	C3	C3	C3
C1	C1	C1	C3	C3	C3

Border Detection as Graph Searching, Region Merging

1. Border Detection as Graph Searching

- Treat pixels as nodes in a graph.
- Edges connect neighboring pixels.
- Finding paths or boundaries where pixel properties change sharply (e.g., intensity).
- Graph search algorithms (like DFS or BFS) find connected borders.

2. Region Merging

- Start with small regions (e.g., each pixel or small groups).
- Merge adjacent regions based on similarity criteria (color, intensity, texture).
- Stop merging when similarity condition fails.

Border Detection as Graph Searching, Region Merging

Example:
Consider a 5×5 binary image:

0	0	0	0	0
0	1	1	0	0
0	1	1	0	0
0	0	0	0	0
0	0	0	0	0

•1 = object pixel
•0 = background pixel

Step 1: Represent pixels as graph nodes

- Each pixel is a node.
- Connect nodes with edges if they are neighbors (4- or 8-connectivity).
- Here, use 4-connectivity (up, down, left, right).

Step 2: Define border pixels

- Border pixels are object pixels adjacent to at least one background pixel.
- Find such pixels by checking neighbors.

Feature-Based Segmentation (Clustering Using Pixel Intensity)

Step 3: Identify border pixels manually

Check each 1-pixel's neighbors:

- Pixel (2,2): neighbors → (2,1)=0 (background), so (2,2) is border.
- Pixel (2,3): neighbors → (2,4)=0, border.
- Pixel (3,2): neighbors → (4,2)=0, border.
- Pixel (3,3): neighbors → (3,4)=0, border.

Step 4: Use graph search (e.g., DFS) to trace border

- Start from a border pixel, say (2,2).
- Visit connected border pixels using DFS.
- Path found: (2,2) → (2,3) → (3,3) → (3,2).

Feature-Based Segmentation (Clustering Using Pixel Intensity)

Example: Same 5×5 grayscale image:

50	**52**	200	205	210
48	**51**	202	208	212
47	**49**	198	204	207
10	**12**	**15**	**13**	**14**
11	**14**	**10**	**12**	**13**

Step 1: Start with individual pixels as regions.

Step 2: Merge regions if pixel intensity difference ≤ 5.

Step 3: Merge adjacent pixels meeting the condition:

- Top-left block: pixels 48-52 → merge to one region.
- Top-right block: pixels 198-212 → merge to second region.
- Bottom block: pixels 10-15 → merge to third region.

Step 4: Resulting regions:

50	**52**	200	205	210
48	**51**	202	208	212
47	**49**	198	204	207
10	**12**	**15**	**13**	**14**
11	**14**	**10**	**12**	**13**

Feature-Based Segmentation (Clustering Using Pixel Intensity)

50	52	200	205	210
48	51	202	208	212
47	49	198	204	207
10	12	15	13	14
11	14	10	12	13

Step 5: Stop merging since inter-region differences > 5.

Region	Pixels	Average Intensity
1	(1,1),(1,2),(2,1),(2,2),(3,1),(3,2)	~50
2	(1,3)-(1,5),(2,3)-(2,5),(3,3)-(3,5)	~205
3	(4,1)-(4,5),(5,1)-(5,5)	~12

Step	Description
Border Detection	Use graph search to find boundary pixels
Region Merging	Merge neighboring pixels/regions based on similarity

Region Splitting and Merging

- **Splitting:** Recursively divide image into smaller regions if a region is not homogeneous (doesn't meet a criterion).
- **Merging:** Combine adjacent regions that are similar.
- **Target:** Segment image into regions of similar characteristics (e.g., intensity).

Step 1: Given Image (4×4 grayscale pixels)

50	52	200	202
49	51	198	205
53	54	201	199
48	50	202	203

Step 2: Define Homogeneity Criterion

•A region is homogeneous if intensity range ≤ 10 (max - min ≤ 10).

Region Splitting and Merging

Step 3: Tial Region — Whole Image

- Check intensity range of full image:
- Min = 48, Max = 205, Range = 205 - 48 = 157 > 10 → Not homogeneous → Split

50	52	200	202
49	51	198	205
53	54	201	199
48	50	202	203

Step 4: Split Image into 4 Quadrants (2×2 blocks)

1. Top-left (TL):

50	52
49	51

Range = 52 - 49 = 3 ≤ 10 → Homogeneous

50	**52**	200	202
49	**51**	198	205
53	54	201	199
48	50	202	203

Region Splitting and Merging

1.Top-right (TR):

Range = 205 - 198 = 7 ≤ 10 → Homogeneous

2. Bottom-left (BL):

Range = 54 - 48 = 6 ≤ 10 → Homogeneous

3. Bottom-right (BR):

Range = 203 - 199 = 4 ≤ 10 → Homogeneous

50	52	200	202
49	51	198	205
53	54	201	199
48	50	202	203

200	202
198	205

53	54
48	50

201	199
202	203

Step 5: Check if any further splitting is needed

- All quadrants are homogeneous, so no further splitting.

Region Splitting and Merging

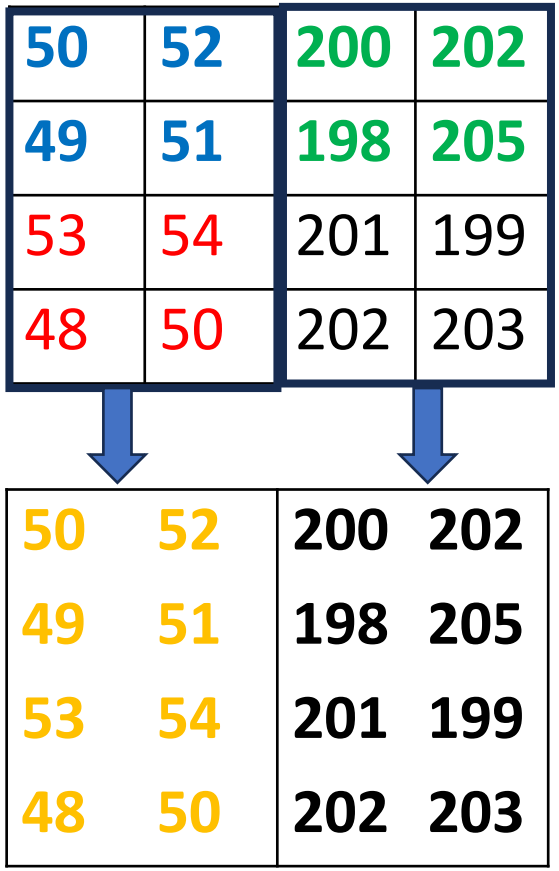
Step 6: Region Merging

Check TL and BL:

- Combined intensities = [50, 52, 49, 51, 53, 54, 48, 50]
- Range = 54 - 48 = 6 ≤ 10 → Merge TL and BL

Check TR and BR:

- Combined intensities = [200, 202, 198, 205, 201, 199, 202, 203]
- Range = 205 - 198 = 7 ≤ 10 → Merge TR and BR



Region Splitting and Merging

Check merged left vs merged right regions:

- Combined intensities = [48, 49, 50, 51, 52, 53, 54, 198, 199, 200, 201, 202, 203, 205]
- Range = 205 - 48 = 157 > 10 → Do NOT merge left and right

Final Regions After Splitting and Merging:

- Left region: pixels with intensities ~48 to 54 (left half)
- Right region: pixels with intensities ~198 to 205 (right half)

Thank You

Lecture 5.8

Image Segmentation

Watershed Algorithm

Instructor Name:

Prof. Dr. Jahangir Alam

Dept. of CSE, BAUST

Watershed Algorithm for Image Segmentation

Watershed Algorithm is one of the most *intuitive and powerful techniques* for image segmentation, especially when you want to separate overlapping or touching objects (like cells, stones, or regions in medical or microscopy images).

Intuitive Concept: "Catchment Basins and Watershed Lines"

Imagine your grayscale image as a topographic surface:

- ✓ Pixel intensity = height
- ✓ Dark areas = valleys (low elevation)
- ✓ Bright areas = peaks (high elevation)

Watershed Algorithm for Image Segmentation

Now imagine “pouring water” into this landscape:

- Water fills the valleys (low-intensity regions).
- As the water level rises, different basins/sink expand.
- When water from different basins is about to merge, you build a dam.
- These dams (boundaries) become the segmentation lines (edges).

Hence the name: Watershed algorithm (like dividing ridges between rivers).

Think of the image as a topographic map:

- Bright = high elevation (ridges or boundaries)
- Dark = low elevation (valleys or basins)
- The algorithm "floods" the image starting from marker points, growing regions until they meet.

Watershed Algorithm for Image Segmentation

Step	Operation	Description
1	Compute Gradient Magnitude	Use Sobel or Canny to highlight edges
2	Identify Markers	Choose starting points for flooding (e.g., by thresholding or distance transform)
3	Simulate Flooding	Flood basins from the markers, level by level
4	Merge Control	Build “dams” where floods meet (watershed lines)
5	Label Regions	Each basin becomes a distinct segment

Watershed Algorithm for Image Segmentation

Mathematical Formulation

Let $I(x, y)$ be a grayscale image. Define gradient magnitude:

$$G(x, y) = \sqrt{(I_x)^2 + \left(I_y\right)^2}$$

This gradient image acts as the topographic map — high gradients = ridges, low gradients = valleys.

The Watershed transform partitions the image into regions R_1, R_2,..., R_n, such that:

- Each region corresponds to a "catchment basin".
- Boundaries between regions correspond to "watershed lines".

Watershed Algorithm for Image Segmentation

Example: Let's use a small grayscale 5×5 image:

5	5	5	5	5
5	3	3	3	5
5	3	1	3	5
5	3	3	3	5
5	5	5	5	5

Interpretation:

- Center pixel (1) = valley (dark)
- Outer pixels (5) = high ground

Step 1: Gradient (High at Borders): Compute gradient roughly,

✓ Low (center) → High (border),

✓ Edges form around the valley.

Step 2: Marker Initialization: Let's mark,

- Lowest intensity (1) = Marker A (inside valley)
- Background (5) = Marker B

Watershed Algorithm for Image Segmentation

Step 3: Flooding Simulation

Start filling from Marker A (valley) upward:

Thus, the "dam" (watershed line) forms at the ring of intensity = 5.

Level	Pixels filled	Action
1	Center (1)	Basin A starts
2	3's around it	Basin expands
3	When water reaches 5's	Boundary (ridge) formed

Step 4: Segmentation Result

The watershed boundary perfectly outlines the central low-intensity object.

Pixel Value	Region
1–3	Region A (basin)
5	Watershed boundary

Watershed Algorithm for Image Segmentation

Before:		After:
5 5 5 5 5		— — — — —
5 3 3 3 5		— A A A —
5 3 1 3 5	→ Segmented →	— A A A —
5 3 3 3 5		— A A A —
5 5 5 5 5		— — — — —

"—" = watershed boundary (ridge line)

"A" = one segmented basin region

Watershed Algorithm for Image Segmentation

Advantages vs Disadvantages

Advantages	Disadvantages
Produces closed contours	Sensitive to noise
No need for edge thresholding	Over-segmentation if not marker-controlled
Works well with morphological preprocessing	Needs good gradient map

Watershed Algorithm for Image Segmentation

The Watershed Algorithm for image segmentation using a small grayscale image. This will help you understand the concept of region growing from markers — like water flooding a topographic surface.

Step 1: Sample Grayscale Image (Values = Heights)

Image I (grayscale intensities):

Note: The central 3×3 region is a valley (darker), likely to be segmented.

	50	50	80	50	50
	50	**10**	30	**10**	50
I =	80	30	0	30	80
	50	10	30	10	50
	50	50	80	50	50

Step 2: Identify Markers (Seeds): Let's assume,

- Label 1 = marker for object A at (1,1)
- Label 2 = marker for object B at (1,3)

Markers:

	0	0	0	0	0
	0	**1**	0	**2**	0
I =	0	0	0	0	0
	0	0	0	0	0
	0	0	0	0	0

Watershed Algorithm for Image Segmentation

Step 3: Flooding Simulation (Low to High): Process pixels from lowest intensity to highest. At each step:

- Grow labeled regions (1 or 2) to neighbors.

If two regions meet → place watershed line (-1).

Start at intensity = 0 → (2,2)
No label yet, surrounded by unlabeled → wait.

	50	50	80	50	50
	50	10	30	10	50
I =	80	30	**0**	30	80
	50	10	30	10	50
	50	50	80	50	50

Intensity = 10 → (1,1), (1,3), (3,1), (3,3)

- (1,1) is already label 1
- (1,3) is already label 2
- (3,1): neighbor to (1,1) ⇒ assign 1
- (3,3): neighbor to (1,3) ⇒ assign 2

		1		2	
I =			**0**		
		1		2	

Watershed Algorithm for Image Segmentation

Step 3: Flooding Simulation (Low to High): Process pixels from lowest intensity to highest. At each step:

- Grow labeled regions (1 or 2) to neighbors.

If two regions meet → place watershed line (-1).

Intensity = 30 → (1,2), (2,1), (2,3), (3,2)

- (1,2) touches 1 & 2 ⇒ conflict → mark as boundary (-1)
- (2,1): touches 1 ⇒ label 1
- (2,3): touches 2 ⇒ label 2
- (3,2): touches 1 & 2 ⇒ boundary (-1)

		1	**30**	2	
$I =$		**30**	**0**	**30**	
		1	**30**	2	

Step 4: Resulting Segmentation Map: Final labels

	0	**0**	**0**	**0**	**0**
	0	1	-1	2	0
$I =$	0	1	0	2	0
	0	1	-1	2	0
	0	0	0	0	0

Watershed Algorithm for Image Segmentation

Output Explanation

- Regions labeled 1 and 2 grew outward from their markers.
- Where they met, watershed lines (-1) were created, acting as boundaries.
- Center pixel (2,2) was unreachable by labels early on and may eventually be filled or remain ambiguous.

Application:

- Medical image segmentation (cells, organs, tumors)
- Object boundary detection
- Separating touching objects (e.g., grains, stones)
- CT / MRI analysis
- Document analysis (character separation)

Watershed Algorithm for Image Segmentation

Summary:

Step	Description
Input	5×5 grayscale image
Markers	Two seeds at low-intensity zones
Growth	Based on intensity (valley → ridge)
Conflict	If 2 labels touch → boundary
Output	Segmented regions + watershed lines

Watershed Algorithm for Image Segmentation

Treats Image as Topographic Surface to Segment Regions

Statement:

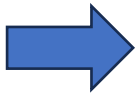
- Imagine the grayscale image as a 3D terrain where pixel intensity represents elevation.
- Watershed lines separate catchment basins (regions).
- Simulate flooding from minima (lowest points), waters from different basins meet at watershed lines.

Watershed Algorithm for Image Segmentation

Treats Image as Topographic Surface to Segment Regions

A small 5×5 grayscale image (pixel intensities represent "elevation"):

10	10	12	50	50
10	11	13	51	52
11	12	14	52	53
50	50	52	60	60
51	52	53	60	61



10	10	12	50	50
10	11	13	51	52
11	12	14	52	53
50	50	52	60	60
51	52	53	60	61

Step 1: Identify local minima (lowest points)

- Lowest intensities: (1,1), (1,2), and (2,1) all with intensity 10.
- These are initial catchment basins (seeds).

Watershed Algorithm for Image Segmentation

Treats Image as Topographic Surface to Segment Regions

Step 2: Flooding simulation — start filling basins from minima upwards

Flood level = 10

- Mark (1,1), (1,2), (2,1) as part of basin A.

10	10	12	50	50
10	11	13	51	52
11	12	14	52	53
50	50	52	60	60
51	52	53	60	61

Flood level = 11

- Include neighbors of basin A pixels with **intensity ≤ 11**: (2,2), (3,1)

- Basin A now includes: {(1,1), (1,2), (2,1), (2,2), (3,1)}

A	A	12	50	50
A	A	13	51	52
A	12	14	52	53
50	50	52	60	60
51	52	53	60	61

Watershed Algorithm for Image Segmentation

Treats Image as Topographic Surface to Segment Regions

10	10	12	**50**	**50**
10	**11**	**13**	51	52
11	12	**14**	52	53
50	**50**	52	60	60
51	52	53	60	61

Flood level = 13

- Include neighbors ≤ 13: (2,3)
- Basin A expands: add (2,3)

Flood level = 14

- Include (3,3) with intensity 14

A	A	A	B	B
A	A	A	B	B
A	A	A	B	B
B	B	B	B	B
B	B	B	B	B

Flood level = 50

- Now, separate higher region starting at (1,4), (1,5), (4,1), (4,2) with intensity 50 — basin B starts here.

Flood level = 51+

- Expand basin B by including adjacent pixels with intensity ≤ current flood level.

Watershed Algorithm for Image Segmentation

Treats Image as Topographic Surface to Segment Regions

Summary:

- Watershed segments image by flooding basins starting from minima.
- Watershed lines separate regions where floods meet.
- Good for segmenting objects with clear intensity valleys.

Clustering-Based Image Segmentation

Uses unsupervised machine learning to group similar pixels.

1. **K-Means Clustering**
   - ✓ Assign each pixel to one of *k* clusters.
   - ✓ Group pixels based on color/intensity similarity.
   - ✓ Minimizes within-cluster variance.
   - ✓ Simple and fast.

2. **Fuzzy C-Means (FCM)**
   - ✓ Similar to K-means but allows partial membership in multiple clusters.
   - ✓ Useful for soft boundaries (e.g., tumors in MRI).
   - ✓ Allows pixels to belong **partially** to multiple clusters — better for soft edges.

Example:
Separating tumor regions in MRI or ultrasound images.

Model-Based Image Segmentation

- Uses geometric or deformable models to fit object boundaries.
- **Active Contours (Snakes)**
  - Curve evolves to fit object boundary by minimizing energy.
  - Energy function:
- $E = E_{internal} + E_{external}$
  - Internal energy → smoothness
  - External energy → edge attraction

- **Level Set Methods**
  - Handles splitting/merging boundaries dynamically.
  - Handles topology changes (e.g., merging regions).

- **Example:**
  Extracting complex organ boundaries in CT or MRI.

Deep Learning-Based Image Segmentation

- Modern & most powerful method
- FCN (Fully Convolutional Networks)
- U-Net (most popular in medical imaging)
- Mask R-CNN for object instance segmentation.

Advantages:

- Learns spatial and texture features automatically from large datasets.
- Produces pixel-level segmentation maps
- Uses CNN for **pixel-level labeling**.

Example:

- Brain tumor or lung nodule or polyp segmentation from CT/MRI.

Deep Learning-Based Image Segmentation

Applications

Domain	Segmentation Purpose
Medical	Organ, tumor, lesion segmentation
Satellite	Land use, vegetation, water bodies
Industrial	Defect detection
Document	Text/line segmentation
Autonomous Vehicles	Road/lane/obstacle detection

Thank You